



FinCoach AI – Personal Finance Intelligence Platform

J.M.N.K. JAYASUNDARA

KUR/IT/2324/F/0054

FinCoach AI – Personal Finance Intelligence Platform

PROJECT REPORT PRESENTED BY

J.M.N.K. JAYASUNDARA

KUR/IT/2324/F/0054

HIGHER NATIONAL DIPLOMA IN INFORMATION TECHNOLOGY

Of the

ADVANCED TECHNOLOGICAL INSTITUTE - KURUNEGALA

SRI LANKA INSTITUTE OF ADVANCED TECHNOLOGICAL EDUCATION (SLIATE)

2026

Declaration

I hereby certify that this project report, *FinCoach AI – Personal Finance Intelligence Platform*, is entirely my own work, carried out under the supervision of Ms. P.G.R.N.J. Gamlath, and has not been submitted, in whole or in part, for any other academic award. Any material drawn from other sources has been fully acknowledged and referenced. This document is the property of the Sri Lanka Institute of Advanced Technological Education (SLIATE), and no part of it may be reproduced or transmitted by any means without the prior written permission of the author.

Signature of Candidate:

Name: J.M.N.K. Jayasundara

Registration Number: KUR/IT/2324/F/0054

Date:

Certified by:

Supervisor Name: Ms. P.G.R.N.J. Gamlath

Date:

Signature:

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, Ms. P.G.R.N.J. Gamlath, for her invaluable guidance, continuous encouragement, and constructive feedback throughout every stage of this project. Her expert advice and patience were instrumental in shaping the direction of FinCoach AI and in helping me overcome the technical and academic challenges encountered during development.

Her support in reviewing my progress, suggesting improvements, and sharing valuable insights on both the technical implementation and the documentation of this project is deeply appreciated. Without her mentorship, the successful completion of this project would not have been possible.

I am truly grateful for the time and effort she dedicated to supervising this project.

Abstract

FinCoach AI is a full-stack, AI-powered personal finance management web application designed to help young professionals, students, freelancers, and multi-income earners take control of their financial lives. Unlike conventional finance applications that merely display transaction histories and static charts, FinCoach AI integrates Anthropic's Claude AI to deliver personalized, conversational financial coaching based on each user's real spending and income data.

The system, built using Next.js 15, TypeScript, PostgreSQL (Neon), and Prisma ORM, allows users to record transactions manually, through AI-powered bill scanning, or by importing CSV/Excel files. It supports multi-role profiles for users with more than one income source, multi-currency transactions with live exchange-rate conversion, savings goal tracking with AI-generated projections, portfolio tracking for personal investments, and automated monthly financial reports delivered via email.

The AI Finance Coach constructs an aggregated monthly financial summary for each user and passes it to the Claude API, enabling the assistant to answer natural-language questions such as "Why did I overspend last month?" with specific, data-driven recommendations rather than generic advice. The application follows an Agile-Incremental development methodology and was implemented as an eight-week Phase 1 Minimum Viable Product, with authentication, transaction management, dashboard analytics, AI chat, goals, portfolio tracking, and automated reporting fully functional and deployed on Vercel.

This report documents the requirement specification, system design, and implementation approach of FinCoach AI, demonstrating how the integration of large language models with traditional personal finance tracking can transform raw financial data into clear, actionable guidance.

Table of Contents

Declaration.....	2
Acknowledgement	3
Abstract.....	4
Table of Figures.....	7
List of Tables	9
List of Abbreviations	10
Chapter 01 – Introduction.....	11
1.1 Purpose	11
1.2 System Overview.....	11
1.3 Problem Statement.....	11
1.4 Goal & Vision.....	12
Chapter 02 – Requirement Specification.....	14
2.1 Requirement Gathering Process	14
2.2 System Functions.....	14
2.3 Software Requirement	15
2.4 Hardware Requirement.....	16
Chapter 03 – Design	17
3.1 Front End Design.....	17
3.2 Database Design	34
3.3 Architectural Design.....	45
Chapter 04 – Implementation	47
4.1 Development Approach.....	47
4.2 Implementation of Key Modules.....	47
4.3 Challenges Faced.....	48
Chapter 05 – Testing.....	50

5.1 Testing Approach	50
5.2 Test Cases	50
5.3 User Acceptance Testing Summary	51
Chapter 06 – Results and Evaluation.....	52
6.1 Achievement of Objectives	52
6.2 Limitations.....	52
Chapter 07 – Conclusion and Future Work	54
7.1 Conclusion	54
7.2 Future Work.....	54
Appendices	56
Appendix A: List of Abbreviations	56
Appendix B: Software Requirements Specification Reference.....	57
References	58

Table of Figures

(Figures appear in Chapter 03 – Design, Sections 3.1 and 3.2)

Figure 3.1: Landing Page	17
Figure 3.2: Sign In Page	18
Figure 3.3: Registration – Step 1: Account Details	18
Figure 3.4: Registration – Step 2: Email OTP Verification	19
Figure 3.5: Registration – Step 3: Account Created	19
Figure 3.6: Forgot Password – Email Entry	20
Figure 3.7: Onboarding – Step 2: Select Primary Profession	20
Figure 3.8: Onboarding – Step 3: Select Income Type	21
Figure 3.9: Onboarding – Step 4: Select Base Currency	21
Figure 3.10: Dashboard – Overview, Charts and AI Insights	22
Figure 3.11: Transactions – List, Filters and Search	23
Figure 3.12: Add Transaction – Manual Entry Tab	24
Figure 3.13: Add Transaction – Bill Scan (AI OCR) Tab	24
Figure 3.14: Add Transaction – File Import Tab	25
Figure 3.15: AI Coach – Conversational Chat Interface	25
Figure 3.16: Savings Goals – Goal Cards with AI Coach Banner	26
Figure 3.17: Savings Goals – Create New Goal	26
Figure 3.18: Savings Goals – Update Progress	27
Figure 3.19: Portfolio – Holdings and AI Risk Insight	28
Figure 3.20: Portfolio – Add Asset	29
Figure 3.21: Financial Reports – Request and History	30
Figure 3.22: Settings – Profile, Roles, Notifications and Plan	31
Figure 3.23: Automated Monthly Report Email	32
Figure 3.24: OTP Verification Email	33
Figure 3.25: Entity Relationship Diagram (ERD)	37
Figure 3.26: Neon Database Console – Table List / Schema Overview	38
Figure 3.27: User Table – Registered Account Record	39

Figure 3.28: Account Table – OAuth/Credential Linkage.....	40
Figure 3.29: Session Table – Active Login Sessions.....	40
Figure 3.30: Transaction Table – Income and Expense Records.....	41
Figure 3.31: Goal Table – Savings Goal Records	41
Figure 3.32: Asset Table – Portfolio Holdings	42
Figure 3.33: ChatMessage Table – AI Coach Conversation History.....	42
Figure 3.34: ReportLog Table – Financial Report Delivery Log	43
Figure 3.35: UserRole Table – Multi-Role Profile Data.....	43
Figure 3.36: UserTypeHistory Table – Role Change Audit Log.....	44
Figure 3.37: Use Case Diagram.....	46

List of Tables

Table 3.1: Primary Key / Foreign Key Relationship Summary	35
Table 5.1: Representative Test Cases	50

List of Abbreviations

The following abbreviations are used throughout this report.

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
ERD	Entity Relationship Diagram
FK	Foreign Key
HND	Higher National Diploma
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
MVP	Minimum Viable Product
OAuth	Open Authorization
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
OTP	One-Time Password
PK	Primary Key
RAG	Retrieval-Augmented Generation
SaaS	Software as a Service
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
SSE	Server-Sent Events
UAT	User Acceptance Testing

Chapter 01 – Introduction

1.1 Purpose

The purpose of this project is to design and develop FinCoach AI, an AI-powered personal finance management web application that helps individuals track income and expenses, manage savings goals, and receive personalized financial advice. The project aims to demonstrate how a conversational large language model (Brown et al., 2020), powered by Anthropic's Claude API (Anthropic, 2024a), can be combined with traditional finance-tracking features to turn raw transaction data into clear and actionable financial guidance, rather than leaving users to interpret numbers and charts on their own.

1.2 System Overview

FinCoach AI is a full-stack SaaS web application built with Next.js 15 (App Router), TypeScript, Tailwind CSS, PostgreSQL (hosted on Neon), and Prisma ORM, and deployed on Vercel (Next.js, 2024; Tailwind Labs, 2024; Neon, 2024; Prisma, 2024; Vercel Inc., 2024b). The system allows a user to register using email/password with OTP verification or Google OAuth, and then set up a profile with a primary profession and up to two secondary roles (for example, a nurse who also does freelance work). Once onboarded, the user can record transactions manually, by scanning a bill using AI image recognition, or by importing a CSV/Excel file.

The application provides a visual dashboard with income-versus-expense charts, category breakdowns, and a savings rate indicator. Users can converse with an AI Finance Coach that uses an aggregated monthly summary of their real data as context, allowing Claude to answer specific questions and provide tailored recommendations. Additional modules include savings goal tracking with AI-generated completion projections, a manual investment portfolio tracker, multi-currency support across 32 currencies with live exchange rates, and automated monthly financial reports sent by email via Gmail SMTP, with an on-demand reporting option available at any time.

1.3 Problem Statement

Young professionals, students, and freelancers commonly struggle to manage their personal finances effectively. Most existing financial applications only display transaction histories and basic charts, leaving users to interpret the data themselves. This frequently results in poor spending habits, low savings, and weak financial planning (Fernandes, Lynch and Netemeyer, 2014; Chen,

Jiang and Li, 2021), because users lack an intelligent system capable of answering practical questions such as "Why did I overspend last month?" or "How can I reach my savings goal faster?"

In addition, many users deal with income and expenses in more than one currency but lack proper tools to manage and report on them seamlessly, and monthly financial summaries are rarely delivered proactively, leaving users unaware of emerging problems until it is too late.

FinCoach AI addresses these problems by combining real-time transaction tracking, multi-currency support, and an AI financial coach that analyses each user's actual monthly data to deliver specific, actionable advice, together with automated monthly reports that keep users informed without requiring them to actively check the application.

1.4 Goal & Vision

The overall goal of FinCoach AI is to democratize personal financial management by combining modern AI capabilities with an intuitive SaaS application. The following objectives were achieved during Phase 1 of the project:

- Developed a fully functional AI-powered financial coaching platform using Next.js 15 and the Claude AI API.
- Enabled users to record, categorize, and manage transactions through manual entry, bill scanning, and file import.
- Implemented a conversational AI chat interface that provides personalized spending insights and goal-based advice.
- Introduced multi-currency support allowing transactions to be recorded and converted in real time across 32 currencies.
- Automated monthly financial report generation and delivery via email, with an on-demand reporting option.
- Provided a savings goal tracking module with AI-generated progress advice and projected completion dates.

The long-term vision for FinCoach AI extends beyond Phase 1 to include Retrieval-Augmented Generation (RAG) for persistent AI memory, Stripe subscription billing, Plaid bank-account

synchronization, and a native mobile application, positioning FinCoach AI as a comprehensive, intelligent personal finance ecosystem.

Chapter 02 – Requirement Specification

2.1 Requirement Gathering Process

The requirements for FinCoach AI were gathered through a combination of personal-experience analysis, competitor research, and academic literature review. As the sole developer, the requirement gathering process began with identifying common pain points in existing personal finance applications such as Mint (Intuit Inc., 2006), YNAB (2004), and Quicken (Quicken Inc., 1983), particularly their reactive nature and lack of personalized guidance.

This was supplemented by reviewing academic literature on large language models in financial advisory contexts (Yang, Liu and Wang, 2023) and on the effectiveness of proactive financial reporting, which validated the aggregated-context AI coaching approach adopted in this project. Requirements were further refined through iterative discussion and feedback sessions with the project supervisor, and were formally documented in a Software Requirements Specification (SRS) following the IEEE 830-1998 standard (IEEE, 1998), covering functional requirements, non-functional attributes, interface requirements, and design constraints.

2.2 System Functions

The key functional requirements of FinCoach AI are summarized below:

- User Authentication & Registration - email/password registration with 6-digit OTP verification, Google OAuth login, and a secure forgot-password flow.
- Multi-Role Profile Management - users may assign a primary profession and up to two secondary roles, with all income and expenses tracked together and role tags used only as AI context.
- Transaction Management - add income/expense transactions manually, via AI-powered bill scanning (OCR), or through CSV/Excel file import, with filtering and search.
- Multi-Currency Support - support for 32 major currencies with live exchange rates and automatic conversion to a user-selected base currency.
- Dashboard & Analytics - visual overview of total income, expenses, and net savings, with income-versus-expense trend charts and category breakdowns.

- AI Coach (Chat) - a conversational interface where users ask natural-language questions about their finances, answered by Claude AI using an aggregated monthly context.
- Savings Goals - creation and tracking of financial goals with AI-generated completion projections and progress bars.
- Portfolio Tracking - manual tracking of investment assets (stocks, crypto, fixed deposits, commodities) with profit/loss calculation.
- Financial Reports - automated monthly report emails and on-demand report requests for any past month.

2.3 Software Requirement

The following software, frameworks, and development tools were used to build FinCoach AI:

- Next.js 15 (App Router) (Next.js, 2024) - full-stack React framework for the frontend and backend API routes.
- TypeScript - statically typed programming language used throughout the codebase.
- Tailwind CSS (Tailwind Labs, 2024) and shadcn/ui (shadcn/ui, 2024) - utility-first styling and pre-built accessible UI components.
- Prisma ORM (Prisma, 2024) with PostgreSQL (Neon, 2024) - type-safe database access and schema management.
- NextAuth.js v5 (NextAuth.js, 2024) - authentication library supporting Google OAuth and credential-based login with JWT sessions.
- Anthropic Claude API (Anthropic, 2024a) and Vercel AI SDK (Vercel Inc., 2024a) - AI coaching, bill-scanning OCR, and streamed chat responses.
- Nodemailer (Nodemailer, 2024) with Gmail SMTP - transactional email delivery for OTP codes and financial reports, authenticated using a Gmail App Password (Google, 2024) rather than a regular account password.
- ExchangeRate-API (ExchangeRate-API, 2024) - live currency exchange-rate data.
- Visual Studio Code, Git and GitHub - code editor and version control.
- Vercel (Vercel Inc., 2024b) - cloud hosting, deployment, and cron-job scheduling for automated monthly reports.

2.4 Hardware Requirement

As a web-based application, FinCoach AI does not require dedicated hardware; however, the following minimum hardware is recommended for optimal usage:

- Client device: a desktop, laptop, tablet, or smartphone with a modern web browser and a minimum display width of 320px.
- Minimum 1 GB of RAM on the client device for smooth browser operation.
- A stable internet connection, required for currency-rate fetching, AI chat, and all other online features.
- A camera (optional) on the client device, required only for the bill-scanning feature on mobile.
- Development machine: a standard laptop/desktop capable of running Node.js, Visual Studio Code, and a modern browser for local development and testing.

Chapter 03 – Design

3.1 Front End Design

The FinCoach AI user interface follows a clean, professional design philosophy built with Tailwind CSS and shadcn/ui components. The primary brand color is purple (#5B4FE8) set against white and dark navy surface backgrounds with light purple accents. The layout uses a fixed sidebar navigation alongside a main content area and topbar, and the interface is fully responsive down to mobile screen widths. The following figures present the actual implemented screens, captured directly from the deployed application, covering the complete user journey from marketing landing page through authentication, onboarding, and every core feature module.

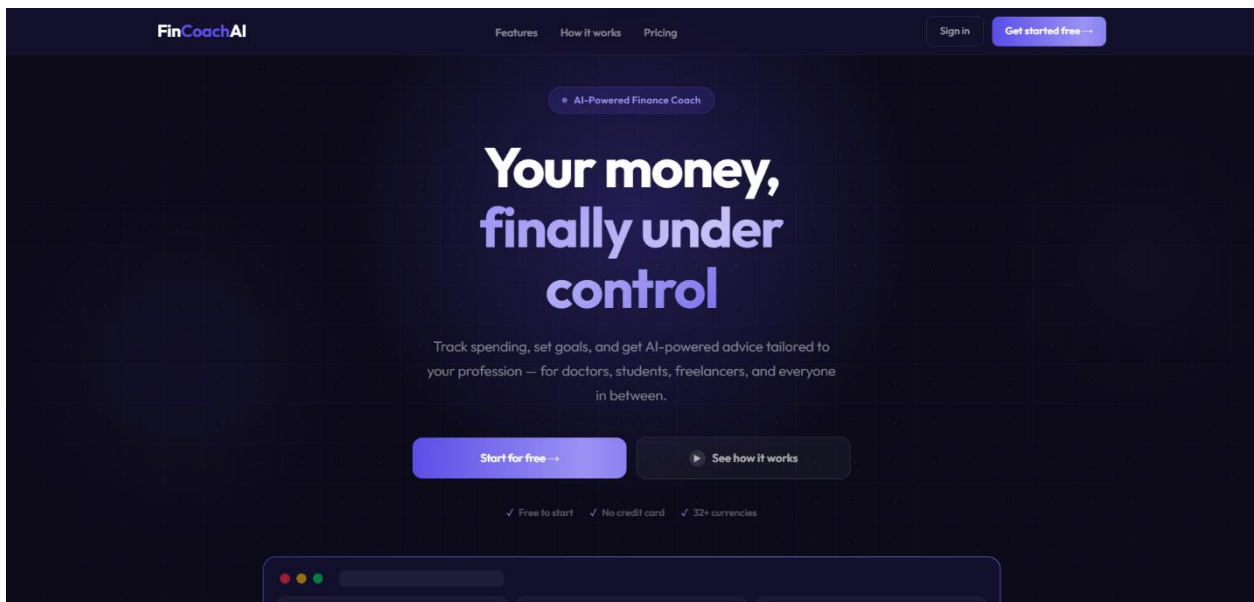


Figure 3.1: Landing Page

This is the very first page people see when they visit the website, before logging in. It has the app name "FinCoachAI" at the top, a heading that reads "Your money, finally under control," and two buttons — "Start for free" and "See how it works." It also lists three trust points: free to start, no credit card needed, and support for 32+ currencies.

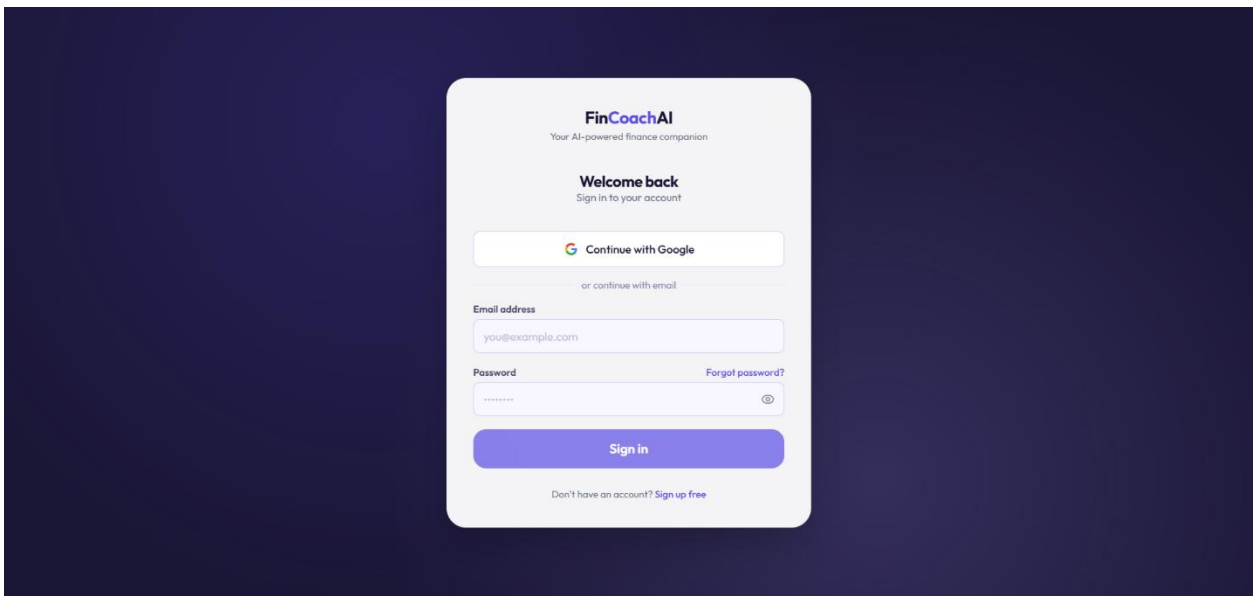


Figure 3.2: Sign In Page

This is the login screen for people who already have an account. They can either click "Continue with Google" to log in instantly, or type their email and password into the boxes below. There is also a "Forgot password?" link and a link to sign up if they do not have an account yet.

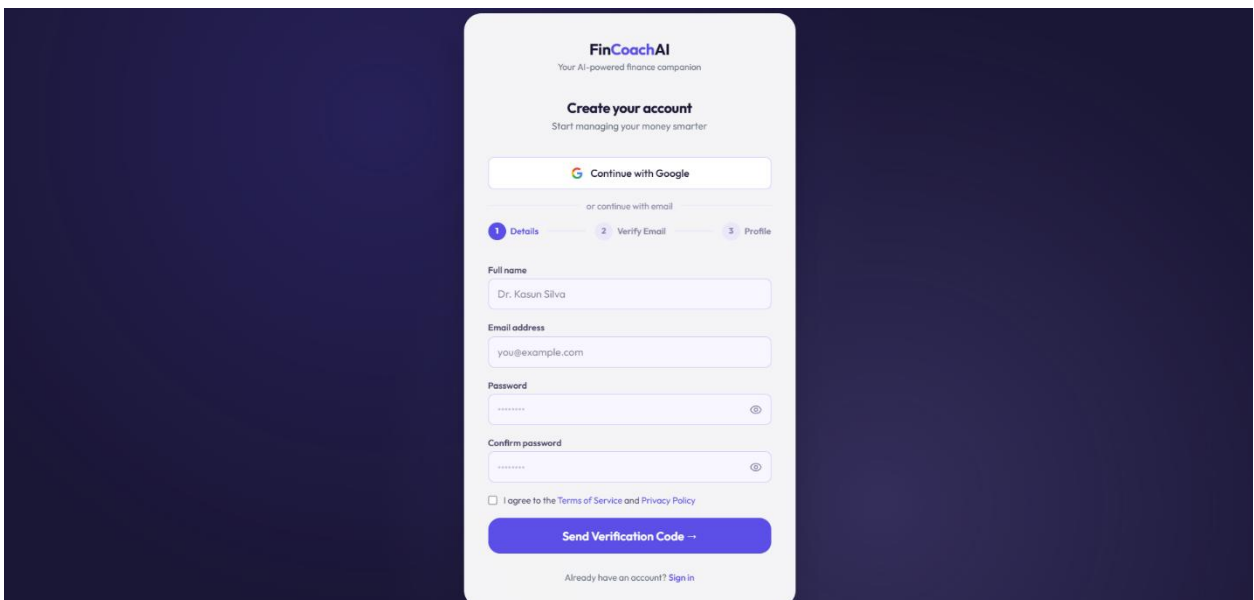


Figure 3.3: Registration – Step 1: Account Details

This is the first step of creating a new account. The user types their full name, email address, and password, and confirms the password. They must also tick a box agreeing to the Terms of Service before continuing. A step indicator at the top shows this is step 1 of 3.

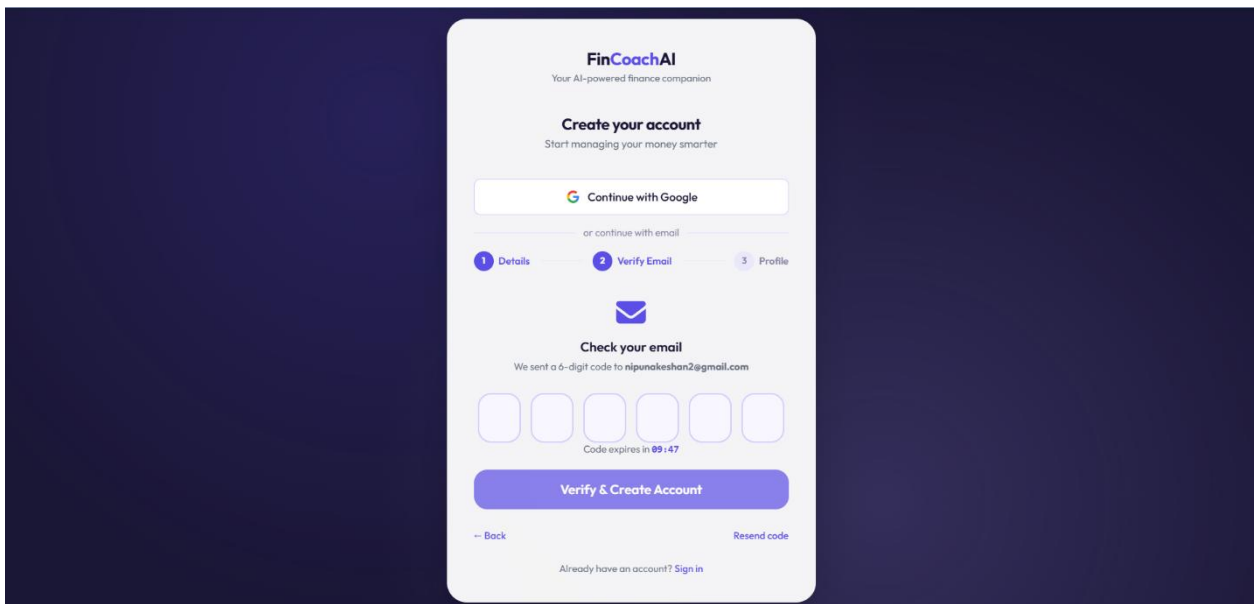


Figure 3.4: Registration – Step 2: Email OTP Verification

After step 1, the system sends a 6-digit code to the user's email. This screen has six small boxes where the user types each digit of that code. There is a countdown timer showing how long the code is valid for, and a "Resend code" option in case it did not arrive.

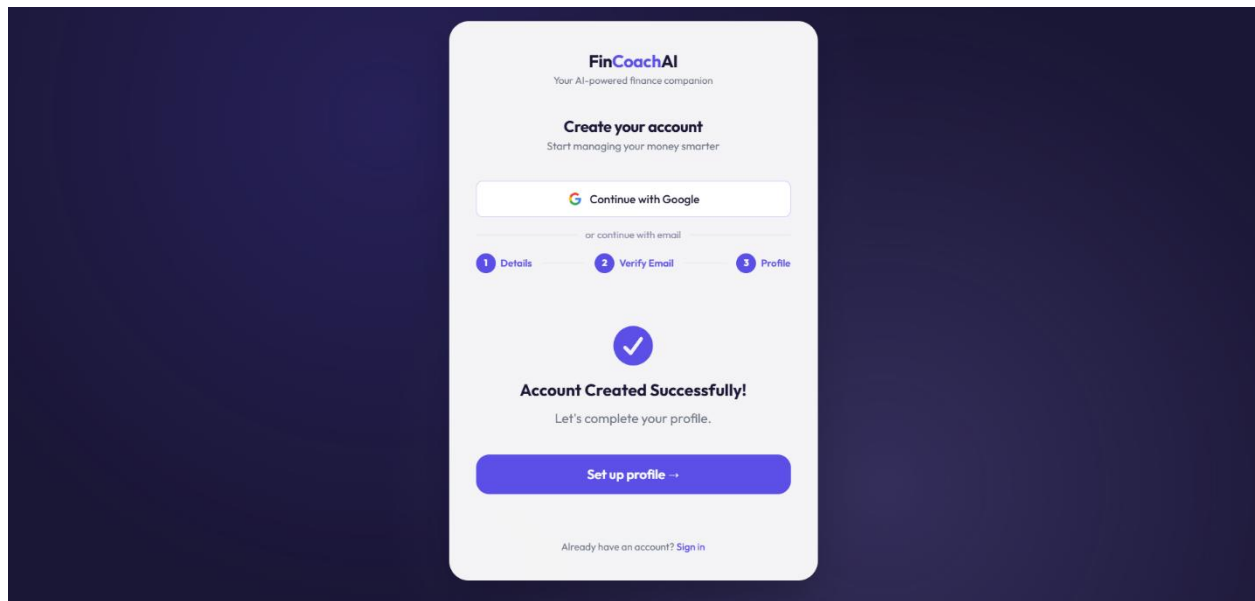


Figure 3.5: Registration – Step 3: Account Created

Once the code is verified correctly, this success screen appears with a checkmark and the message "Account Created Successfully!" It then asks the user to click "Set up profile" to continue.

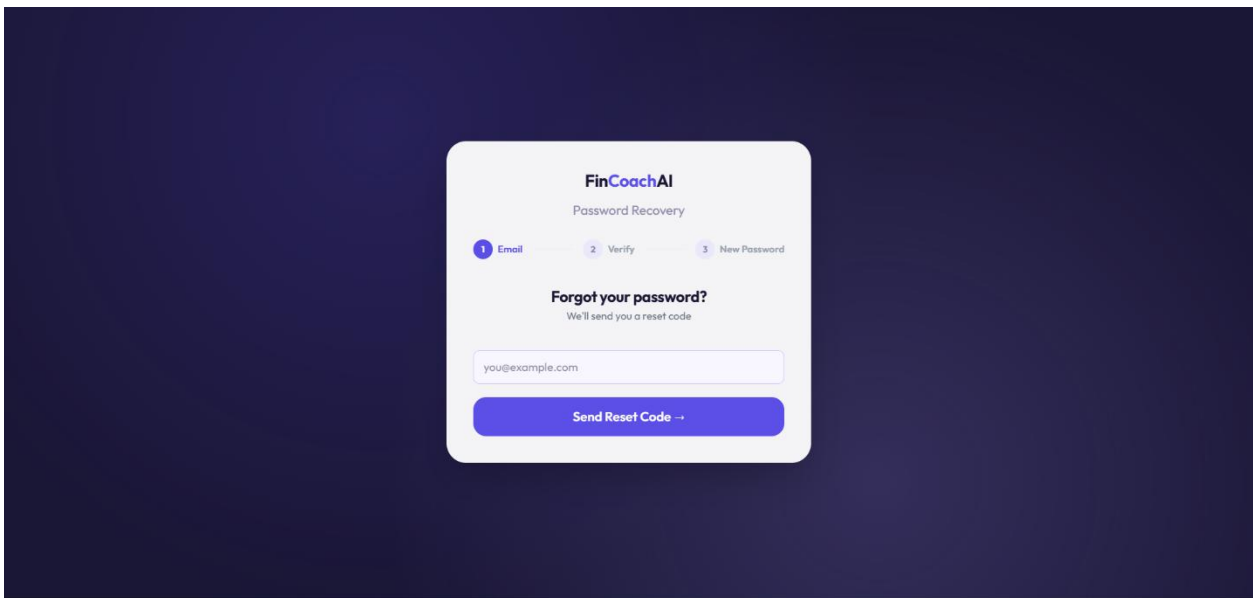


Figure 3.6: Forgot Password – Email Entry

This is the first screen in the password recovery process. The user types the email address linked to their account, and the system sends them a reset code, similar to the registration OTP.

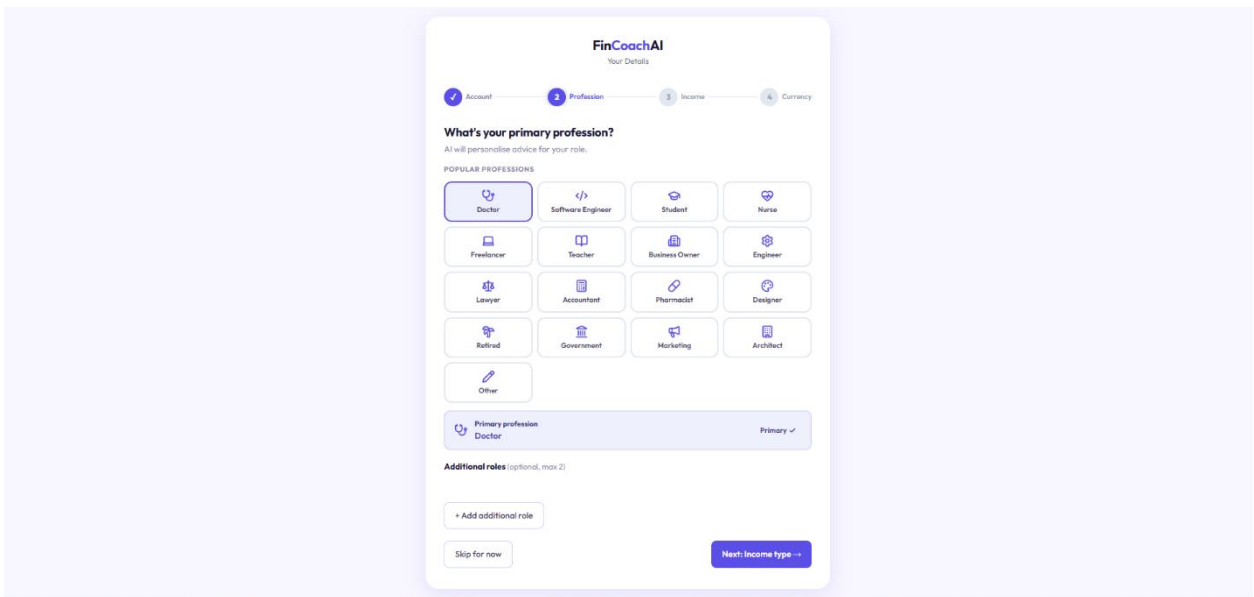


Figure 3.7: Onboarding – Step 2: Select Primary Profession

After creating an account, the user is asked what their main job is (Doctor, Student, Nurse, Freelancer, etc.). This helps the AI give advice suited to their profession. The user can also add up to two extra roles if they have more than one job.

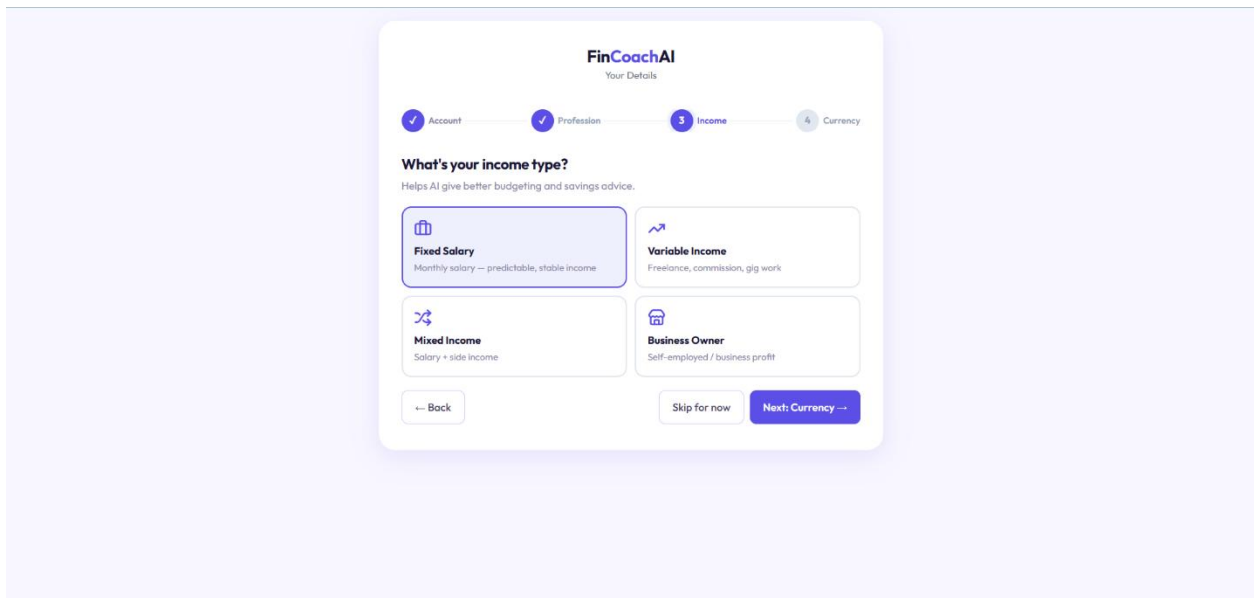


Figure 3.8: Onboarding – Step 3: Select Income Type

This screen asks how the user earns money — a Fixed Salary (steady monthly pay), Variable Income (freelance/commission), Mixed Income (salary plus side income), or Business Owner. This helps the AI understand how stable or unpredictable their income is.

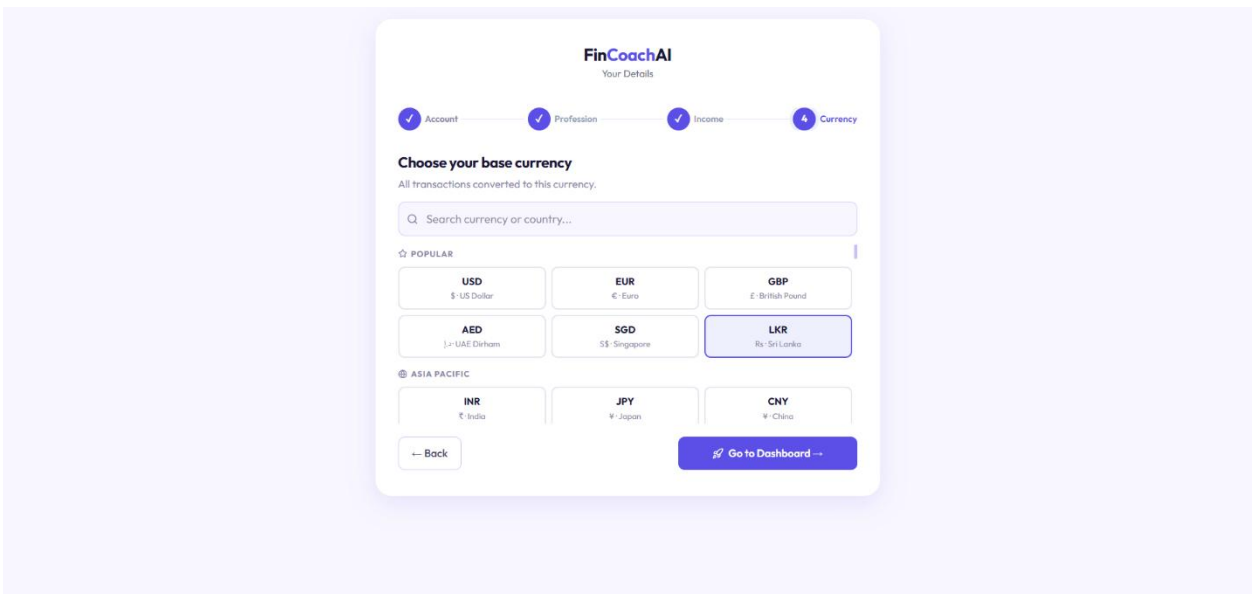


Figure 3.9: Onboarding – Step 4: Select Base Currency

Here the user picks the currency they want all their money shown in (for example, LKR for Sri Lankan Rupees). There is a search box and popular currency options like USD, EUR, GBP, and LKR.

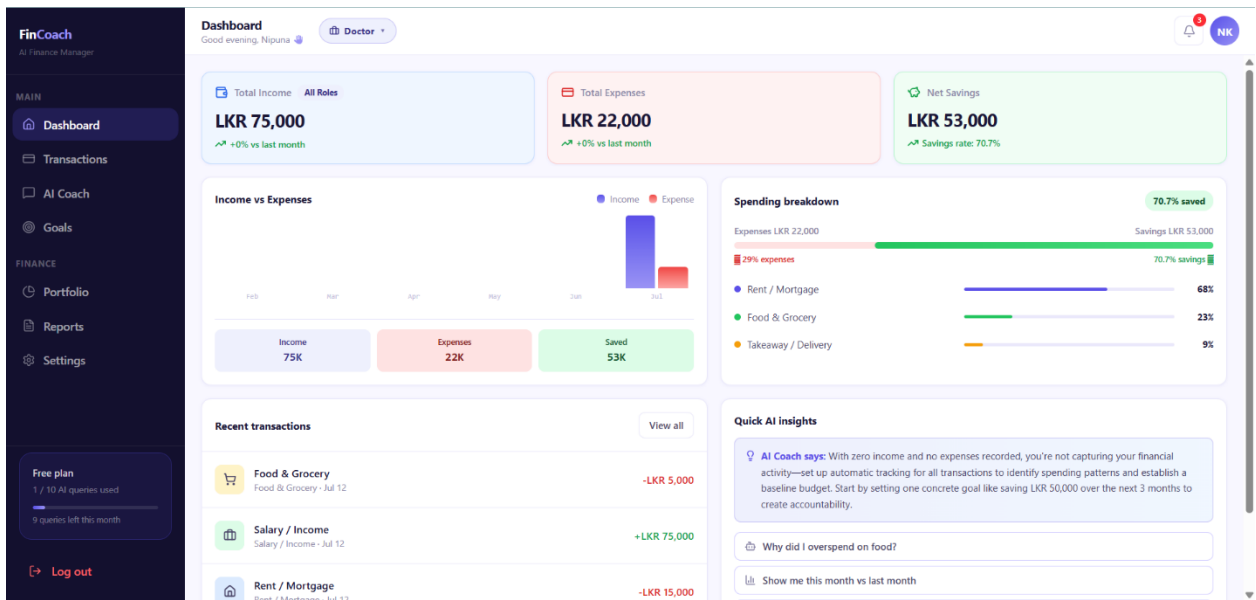


Figure 3.10: Dashboard – Overview, Charts and AI Insights

This is the main home screen after logging in. It shows three summary boxes at the top: Total Income, Total Expenses, and Net Savings. Below that is a bar chart comparing income vs expenses over recent months, a breakdown of spending by category (like Rent, Food, Takeaway), a list of recent transactions, and a box where the AI gives a quick tip about the user's spending.

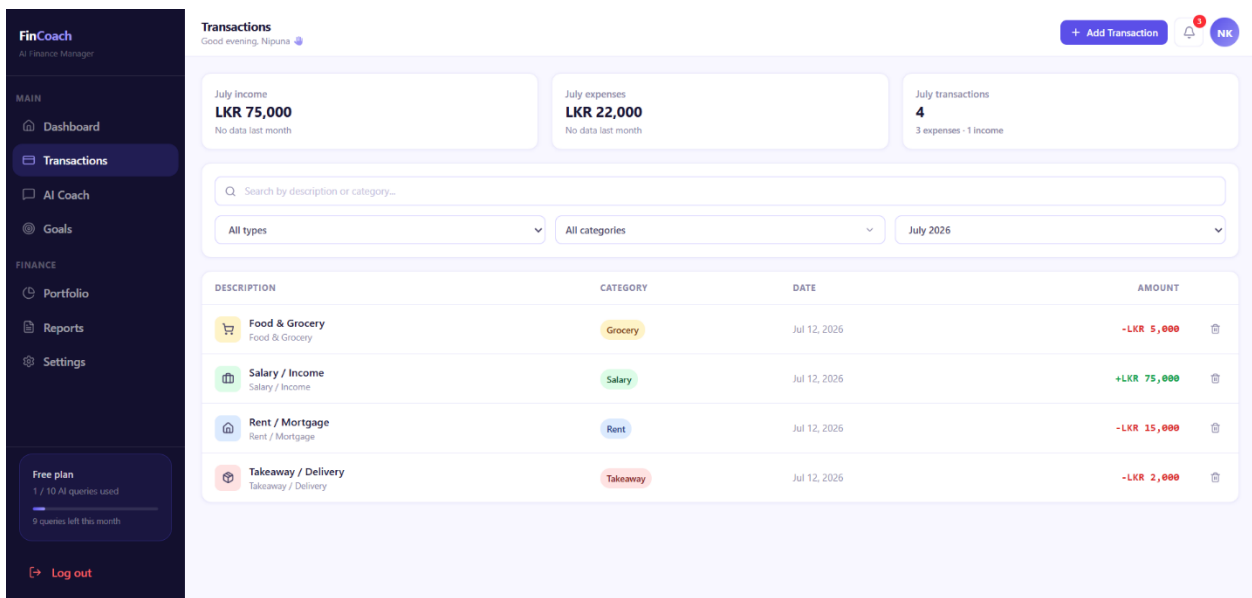


Figure 3.11: Transactions – List, Filters and Search

This page lists every single income and expense record the user has added, in a table with columns for description, category, date, and amount. At the top there are summary cards for the month's income, expenses, and transaction count, plus a search bar and filter dropdowns (by type, category, and month).

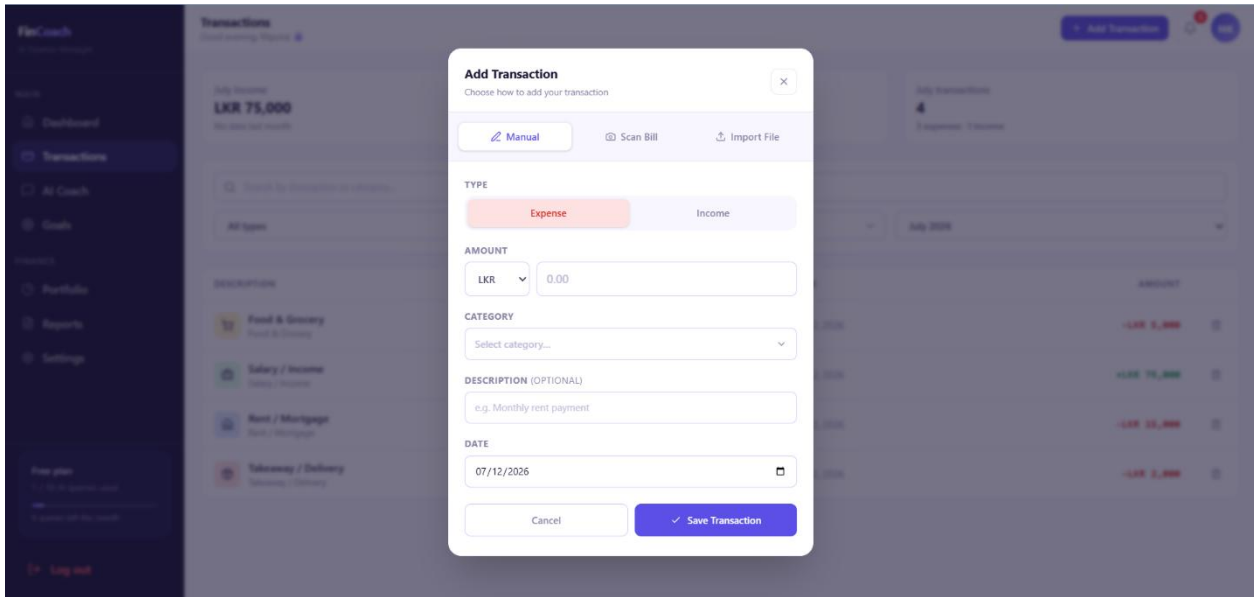


Figure 3.12: Add Transaction – Manual Entry Tab

This is a popup form for adding a transaction by typing it in manually. The user chooses whether it is an Expense or Income, enters the amount and currency, picks a category, optionally adds a description, and sets the date.

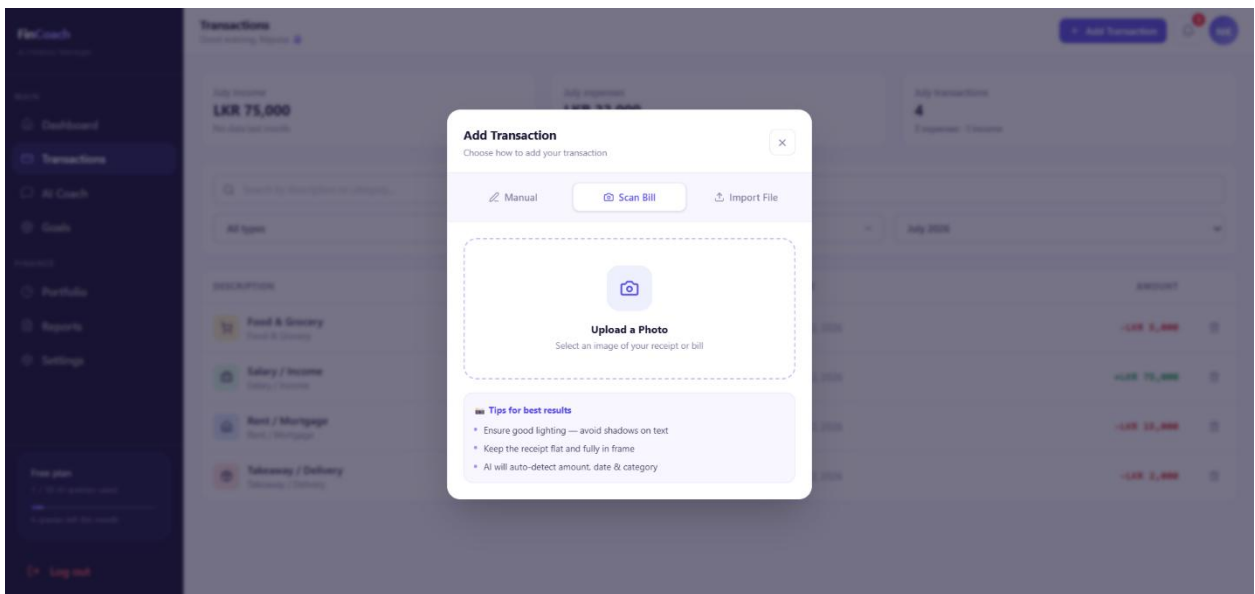


Figure 3.13: Add Transaction – Bill Scan (AI OCR) Tab

Instead of typing everything, the user can upload a photo of a paper receipt or bill. The AI reads the image and automatically pulls out the amount, date, and category. Tips on screen suggest good lighting and keeping the receipt flat for best results.

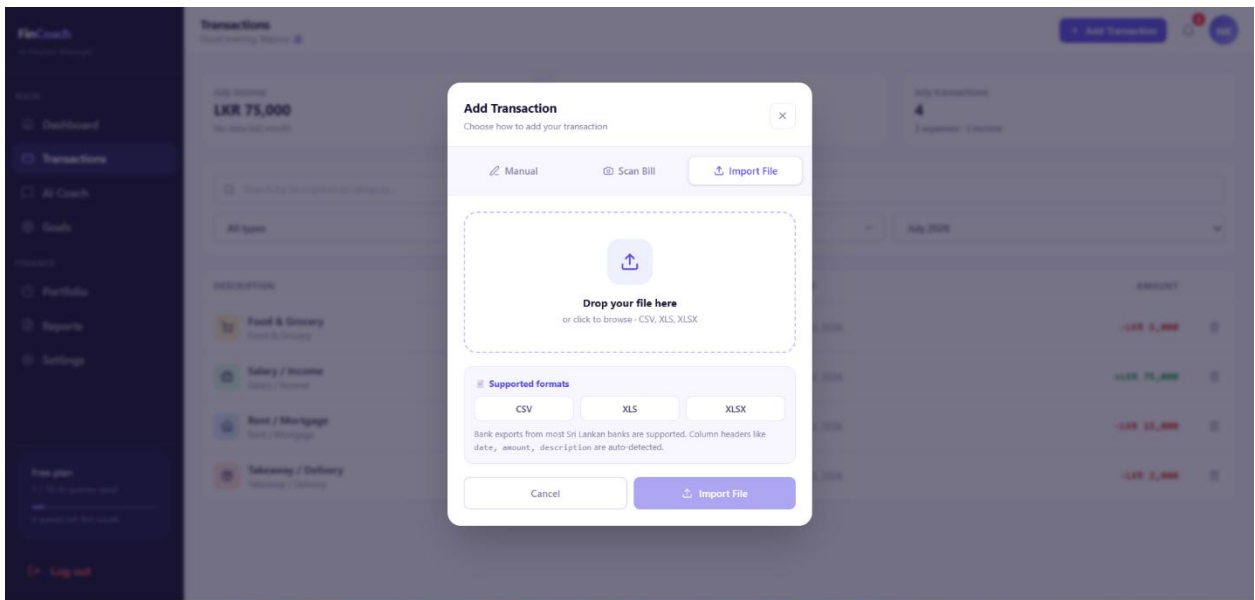


Figure 3.14: Add Transaction – File Import Tab

This tab lets the user upload a CSV or Excel file containing many transactions at once (for example, exported from their bank). The system reads the file and adds all the valid rows automatically.

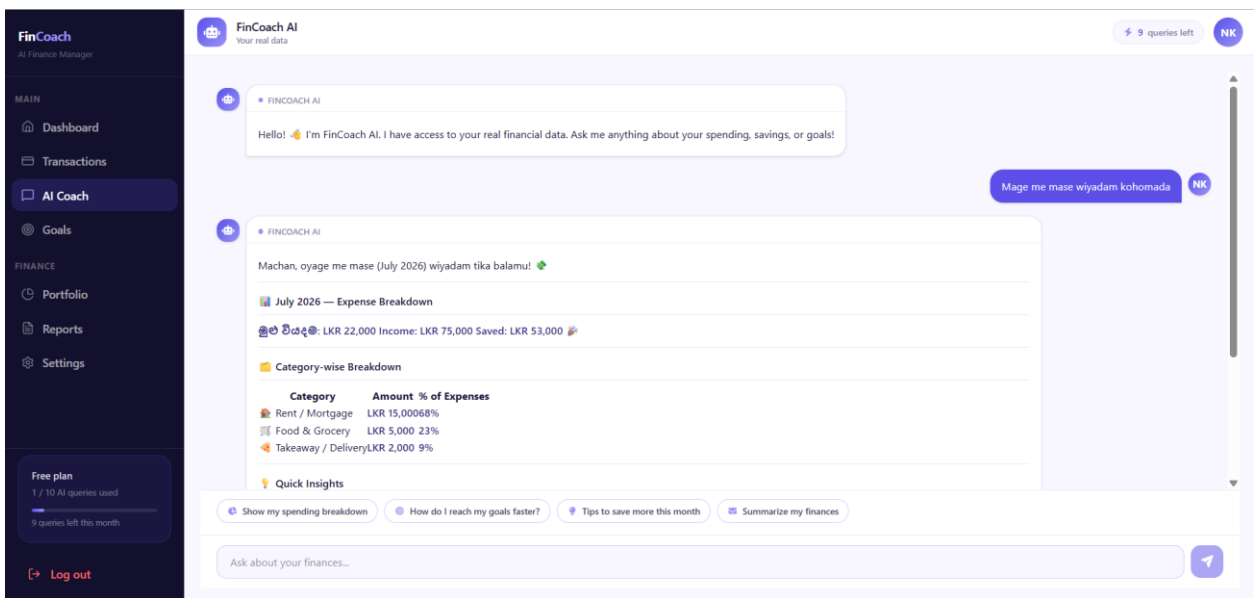


Figure 3.15: AI Coach – Conversational Chat Interface

This is a chat conversation with the AI assistant. The user can type questions like "How is my spending this month?" and the AI replies using the user's real transaction data — showing numbers, percentages, and tips. There are also quick-reply buttons for common questions.

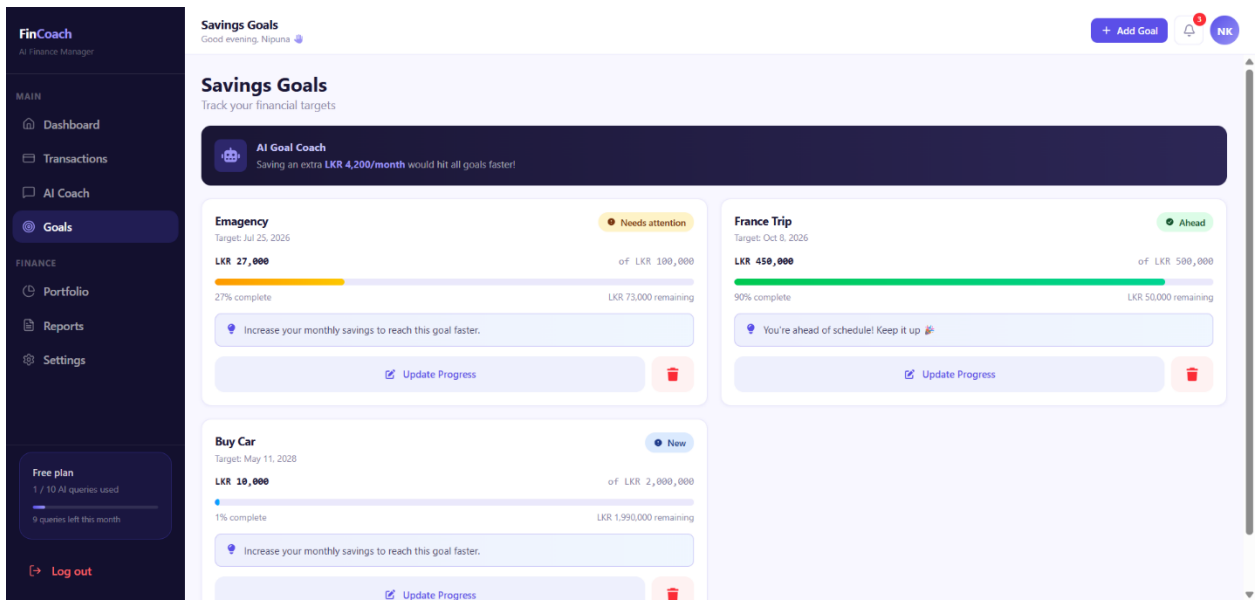


Figure 3.16: Savings Goals – Goal Cards with AI Coach Banner

This page shows all the savings goals the user has created as cards — each with a target amount, how much has been saved so far, a progress bar, and a status badge like "Needs Attention" or "Ahead." There is also a banner at the top where the AI suggests how to reach goals faster.

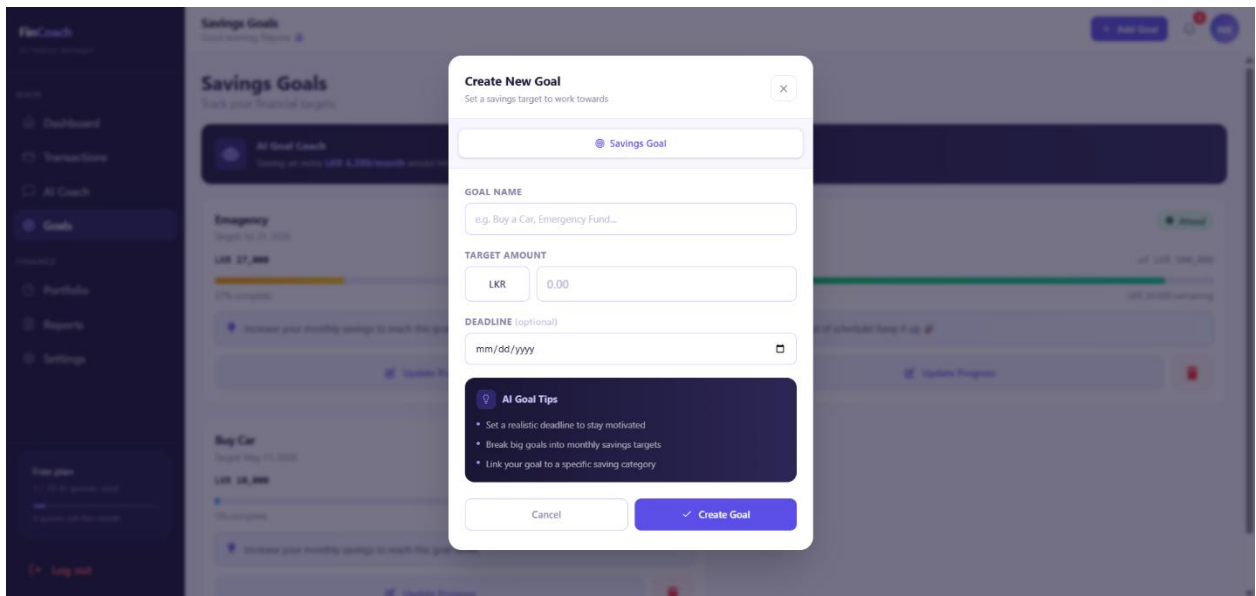


Figure 3.17: Savings Goals – Create New Goal

A popup form for adding a new savings goal. The user types a goal name (like "Buy a Car"), sets a target amount and currency, and optionally picks a deadline date.

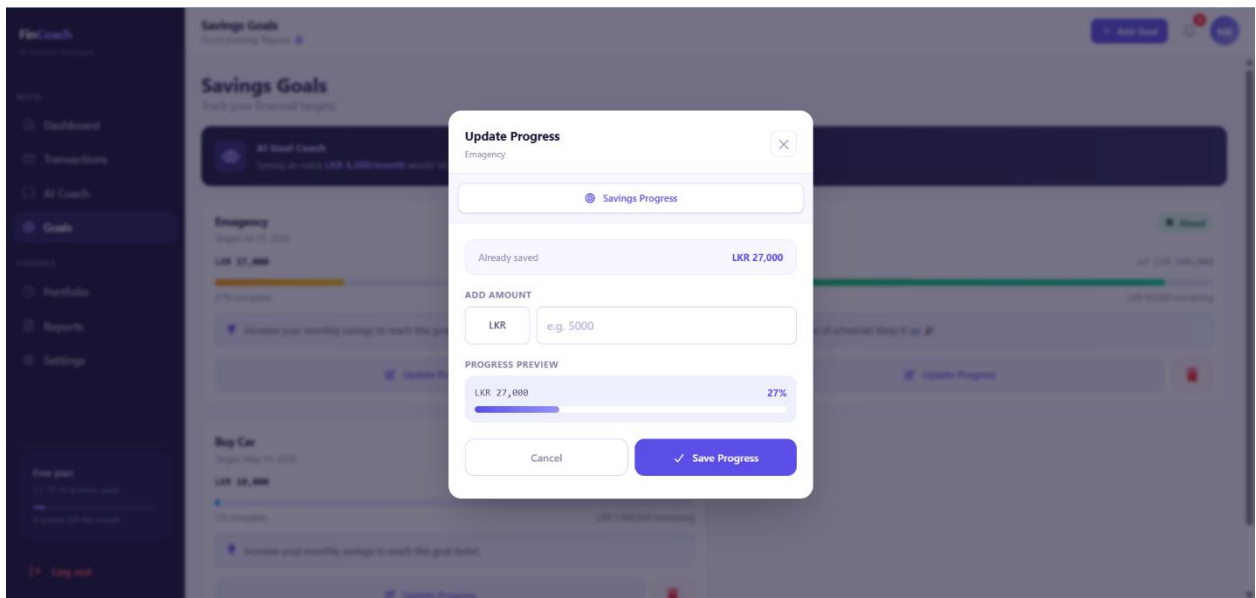


Figure 3.18: Savings Goals – Update Progress

A popup form used to add more money toward an existing goal. It shows how much has already been saved, lets the user type in a new amount to add, and shows a live preview of the updated progress bar before saving.

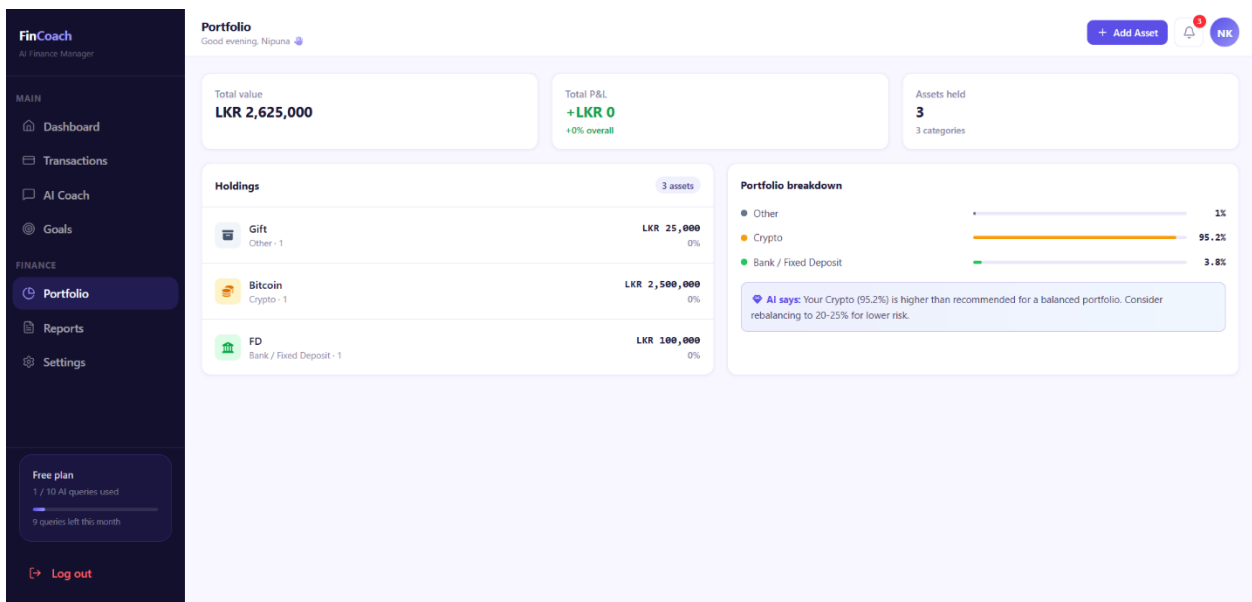


Figure 3.19: Portfolio – Holdings and AI Risk Insight

This page shows the user's investments — like Bitcoin, Fixed Deposits, or Gifts — with their total value and profit/loss. It includes a breakdown chart showing what percentage of the portfolio is in each category, plus an AI comment warning if the investments are too concentrated in one type (like too much in Crypto).

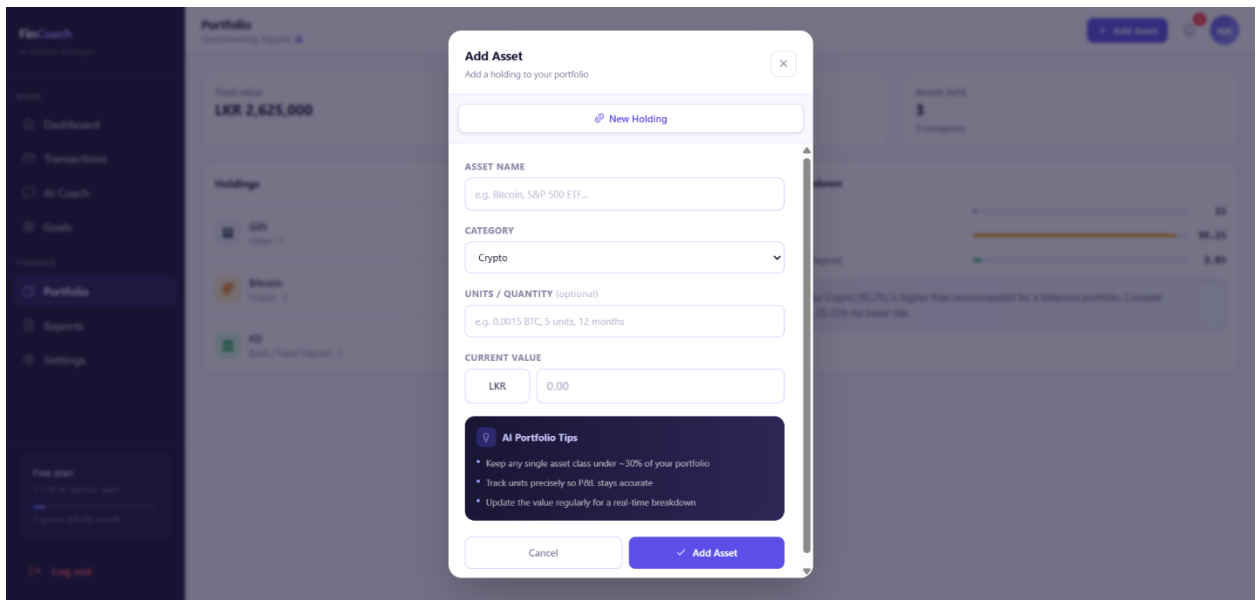


Figure 3.20: Portfolio – Add Asset

A popup form for adding a new investment to the portfolio. The user types the asset name, picks a category (Crypto, Stock, Bank/Fixed Deposit, etc.), optionally enters the quantity, and enters its current value.

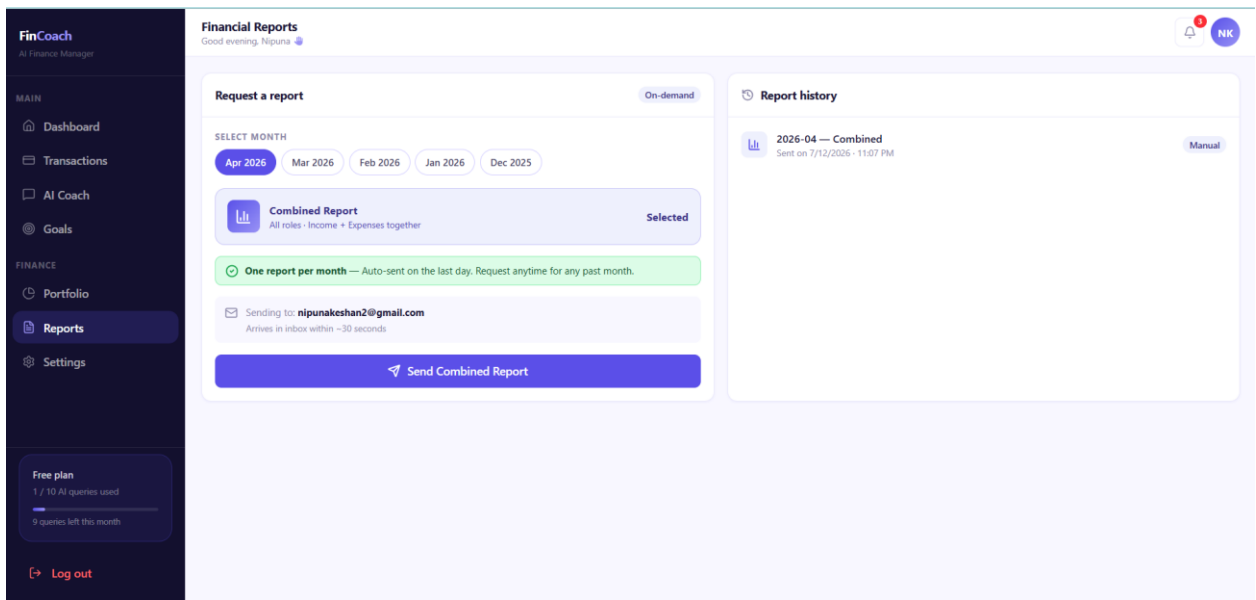


Figure 3.21: Financial Reports – Request and History

This page lets the user request a financial report for any past month by clicking a month button and pressing "Send Combined Report." It also shows a history of reports that were already sent, with the date and time.

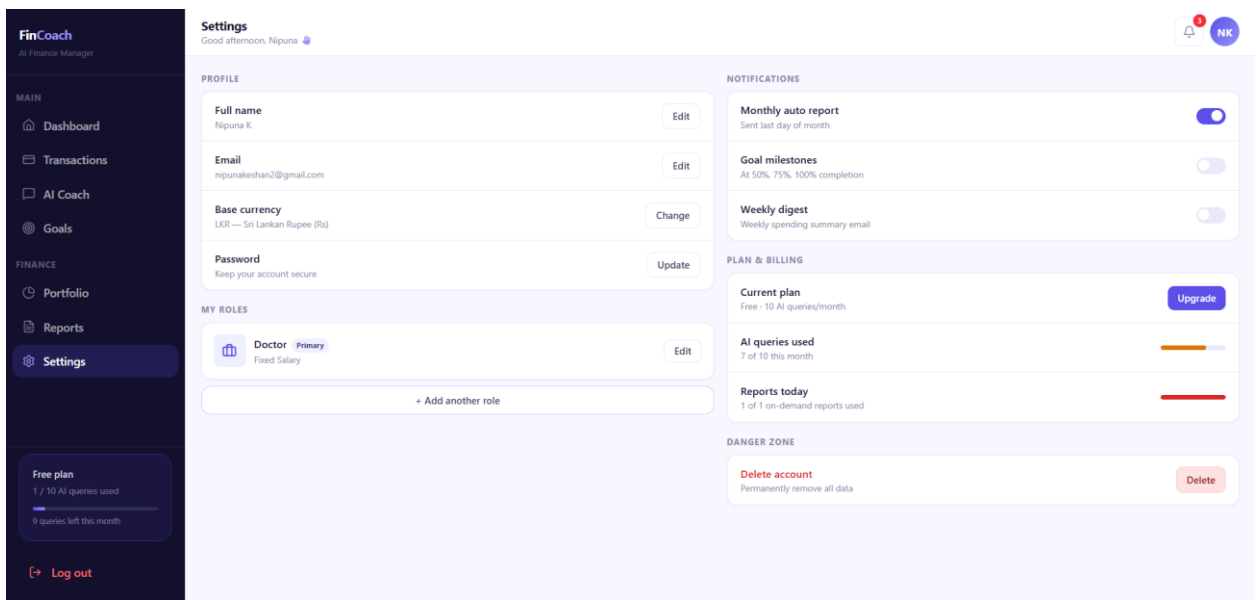


Figure 3.22: Settings – Profile, Roles, Notifications and Plan

This page lets the user manage their account: edit their name/email, change their base currency, update their password, manage their professional roles, turn notification toggles on/off (like monthly reports or spending alerts), see their current plan and AI usage, and delete their account if needed.

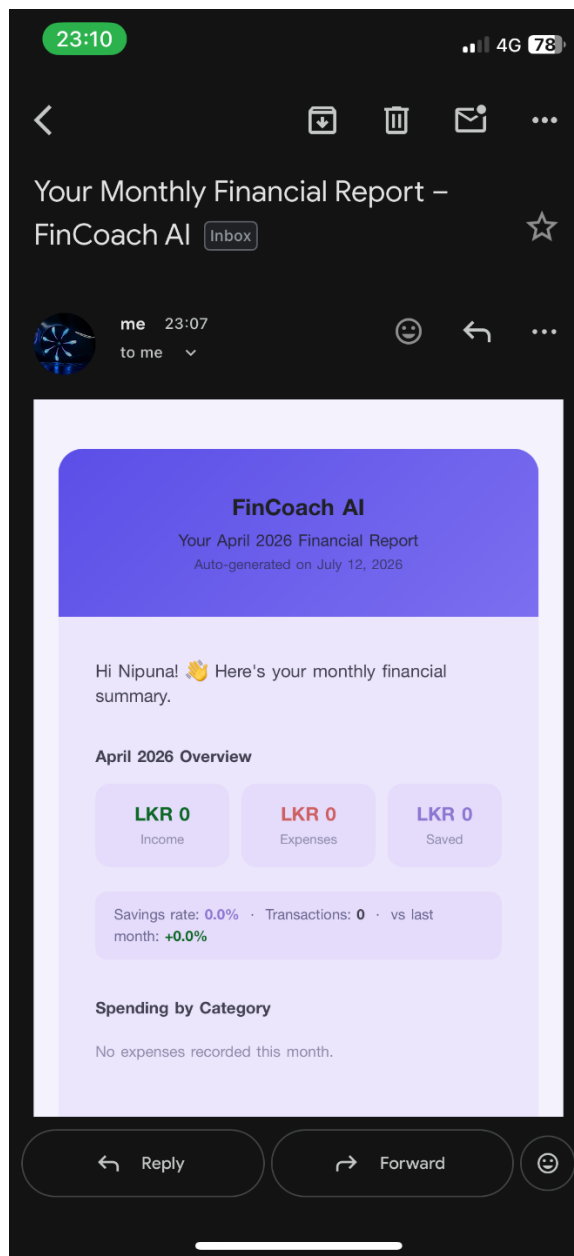


Figure 3.23: Automated Monthly Report Email

This is a screenshot of an actual email the app sent to the user's inbox. It is a summary of their finances for one month — showing income, expenses, savings, and a spending breakdown, formatted with the FinCoach AI branding.

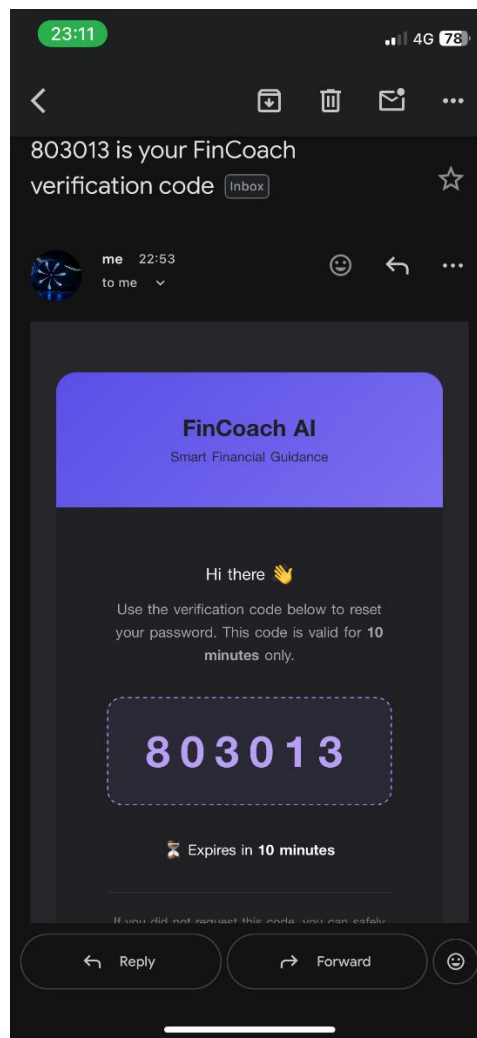


Figure 3.24: OTP Verification Email

This is a screenshot of the actual email containing the 6-digit verification code, sent during registration or password reset. It clearly displays the code in large text and states that it expires in 10 minutes.

3.2 Database Design

FinCoach AI uses a relational PostgreSQL database, hosted on Neon, accessed through Prisma ORM for type-safe queries and schema migrations. The schema is defined declaratively in `schema.prisma` and consists of the following models:

- **User** - the central entity, storing `id`, `name`, `email`, `hashed password`, `image`, `plan`, `primaryProfession`, `secondaryRoles`, `incomeType`, `preferredCurrency`, `onboardingDone`, `password-reset token fields`, and AI usage counters (`aiQueriesUsed`, `aiQueryMonth`, `autoReportEnabled`).
- **Account and Session** - NextAuth.js tables that link a User to an OAuth provider (`type`, `provider`, `providerAccountId`, `tokens`) and store active JWT sessions (`sessionToken`, `expires`).
- **VerificationToken** - stores hashed OTP/reset tokens with an identifier and expiry, uniquely constrained on (`identifier`, `token`).
- **UserRole** - records each professional role name linked to a User, uniquely constrained per (`userId`, `roleName`).
- **UserTypeHistory** - an audit log of role changes (`fromType`, `toType`, `reason`, `changedAt`) linked to a User, used to build AI context.
- **Transaction** - records each income or expense entry: `type`, `amount`, `currency`, `amountBase` (converted value), `category`, `description`, `date`, and `source` (`MANUAL`, `BILL_SCAN`, or `IMPORT`), linked to a User.
- **Goal** - stores savings goals: `name`, `targetAmount`, `currentAmount`, `currency`, `optional deadline`, and `status`, linked to a User.
- **ReportLog** - records every financial report generated: `month` (`YYYY-MM`), `type`, `sentAt`, and `emailSentTo`, linked to a User.
- **ChatSession** - represents one AI Coach conversation thread: `optional title`, `createdAt`, and `updatedAt`, linked to a User.
- **Message** - stores each AI Coach conversation turn (`role`: `user/assistant`, `content`, `tokensUsed`, `createdAt`), linked to a ChatSession rather than directly to a User.

- Asset - tracks a portfolio holding: name, category (Crypto, Stock/ETF, Bank/Fixed Deposit, Commodity, Other), units, value, costBasis (used to calculate profit/loss), and currency, indexed on userId.

Every model other than Account, Session, VerificationToken, and Message carries a userId foreign key with a cascading delete relation back to User, giving User a one-to-many relationship with Account, Session, Transaction, Goal, UserRole, UserTypeHistory, ChatSession, ReportLog, and Asset. Message instead carries a chatSessionId foreign key back to ChatSession, giving User an indirect one-to-many relationship with Message through ChatSession. Table 3.1 below summarises the Primary Key (PK), Foreign Key (FK), and relationship type for every table in the schema.

Table 3.1: Primary Key / Foreign Key Relationship Summary

Table	Primary Key (PK)	Foreign Key (FK)	References	Relationship
User	id	—	—	Root/parent entity — referenced by every other table below
Account	id	userId	User.id	Many Accounts → One User (cascade delete)
Session	id	userId	User.id	Many Sessions → One User (cascade delete)
VerificationToken	(identifier, token)	—	identifier = email	Logical link to User by email; no enforced FK
UserRole	id	userId	User.id	Many UserRoles → One User (cascade delete)
UserTypeHistory	id	userId	User.id	Many History rows → One User (cascade delete)
Transaction	id	userId	User.id	Many Transactions → One User (cascade delete)
Goal	id	userId	User.id	Many Goals → One User (cascade delete)
ReportLog	id	userId	User.id	Many ReportLogs → One User (cascade delete)

Table	Primary Key (PK)	Foreign Key (FK)	References	Relationship
ChatSession	id	userId	User.id	Many ChatSessions → One User (cascade delete)
Message	id	chatSessionId	ChatSession.id	Many Messages → One ChatSession (cascade delete)
Asset	id	userId	User.id	Many Assets → One User (cascade delete)

In short, User is the single parent (root) table, identified by its Primary Key id. Every other table is a child table that stores a userId column as a Foreign Key pointing back to User.id — this is what creates the one-to-many relationships (one User can have many Transactions, many Goals, many Assets, and so on). All of these Foreign Keys use ON DELETE CASCADE, meaning that if a User record is deleted, every related Transaction, Goal, Asset, ChatSession, ReportLog, UserRole, UserTypeHistory, Account, and Session row is automatically deleted with it - and because Message rows cascade from ChatSession rather than directly from User, they are removed as part of the same chain when their parent ChatSession is deleted, keeping the database free of orphaned records. The only exception is VerificationToken, which is not directly linked by a Foreign Key - it is matched to a user logically through the shared email address stored in its identifier column. The conceptual entity relationship diagram is shown in Figure 3.25, followed by screenshots of the live Neon PostgreSQL console confirming that each table exists in production and holds real application data (Figures 3.26–3.36).

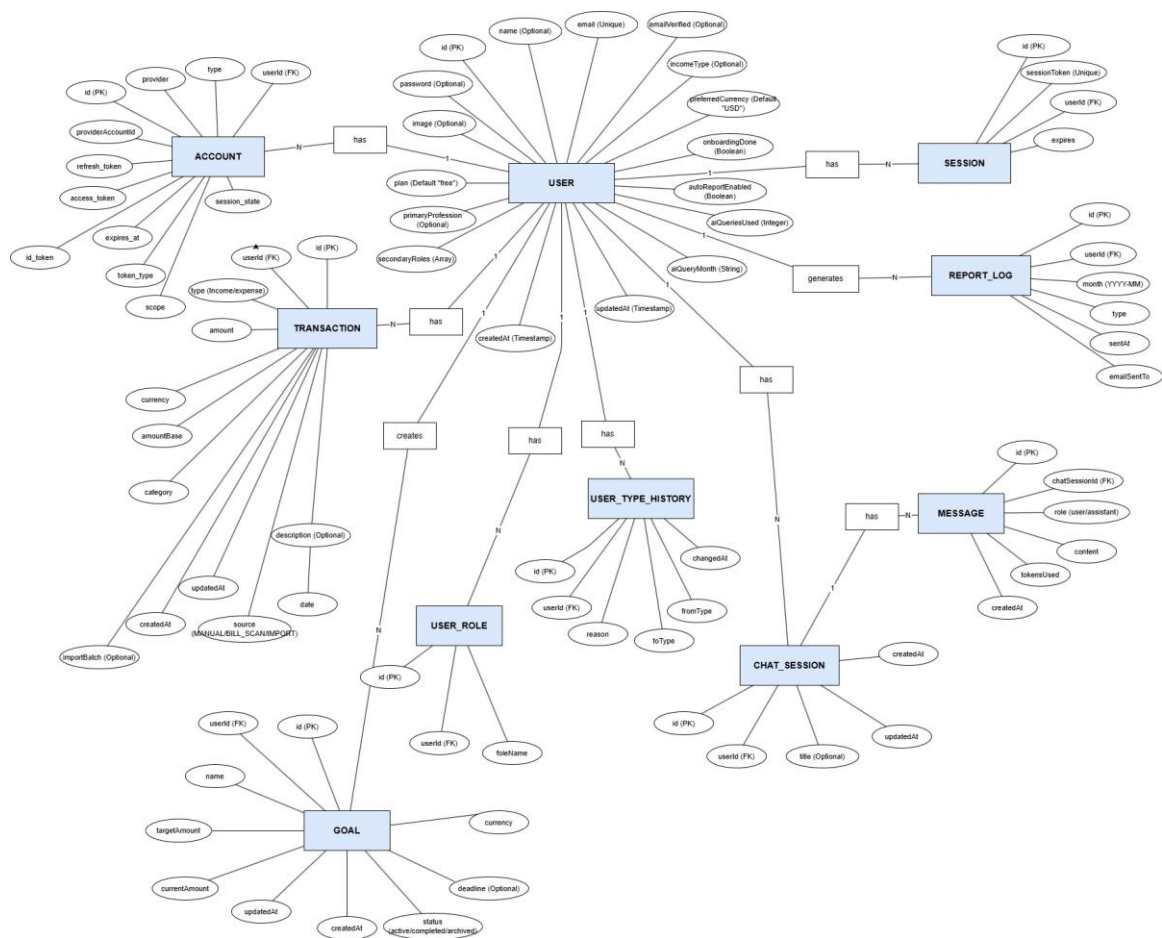


Figure 3.25: Entity Relationship Diagram (ERD)

Conceptual ERD showing the core entities of the FinCoach AI schema and their relationships, derived from the Prisma schema above.

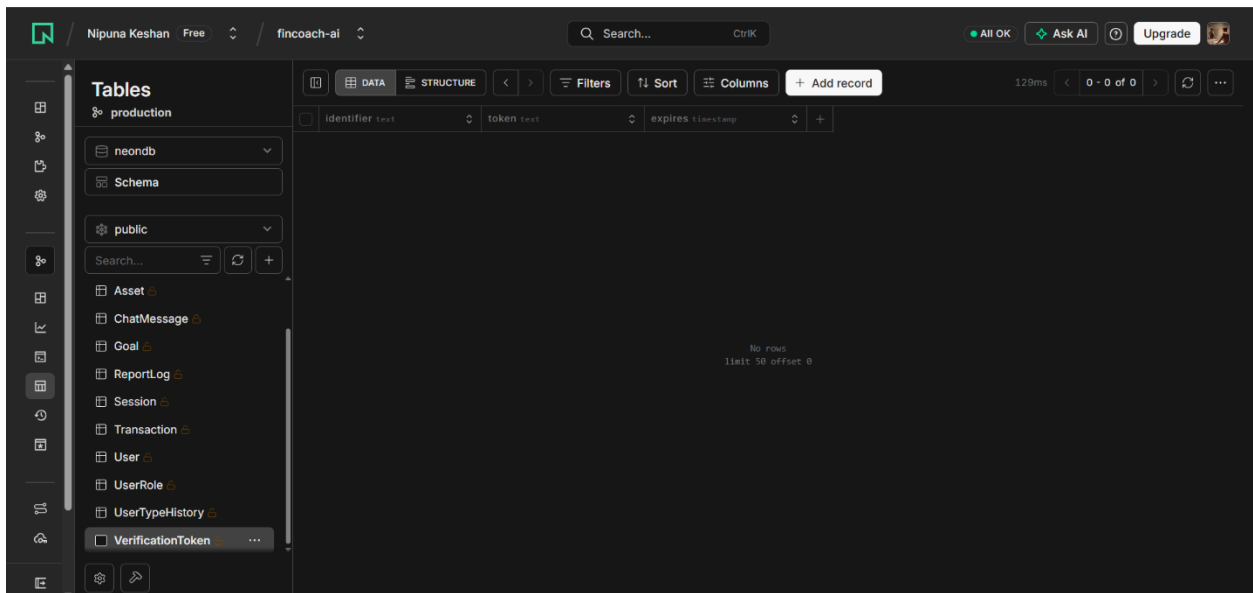


Figure 3.26: Neon Database Console – Table List / Schema Overview

The Neon PostgreSQL console showing the full list of tables that exist in the production database - User, Account, Session, VerificationToken, UserRole, UserTypeHistory, Transaction, Goal, ReportLog, ChatSession, Message, and Asset. This confirms the schema was actually deployed, not just designed on paper.

The screenshot shows a database management interface. On the left, a sidebar lists tables under the 'production' schema: neondb, Schema, public, .prisma_migrations, Account, Asset, ChatMessage, Goal, ReportLog, Session, Transaction, User (selected), and UserRole. The main area displays the 'User' table structure and a single record. The table has columns: id (text), name (text), email (text), emailVerified (timestamp), password (text), and image (text). The record shows a user with a long alphanumeric ID, the name 'Nipuna K Jayasundara', email 'nipunakeshan2@gmail.com', a NULL emailVerified timestamp, a bcrypt-hashed password, and a NULL image.

id	name	email	emailVerified	password	image
cmr12aylm000004js2epmx...	Nipuna K Jayasundara	nipunakeshan2@gmail.com	NULL	\$2b\$12\$uERq67pYsta5ujL...	NULL

Figure 3.27: User Table – Registered Account Record

A live User record showing id (Primary Key), name, email, and bcrypt-hashed password. The User table has no foreign keys of its own - it is the parent table that every other table links back to through a userId column.

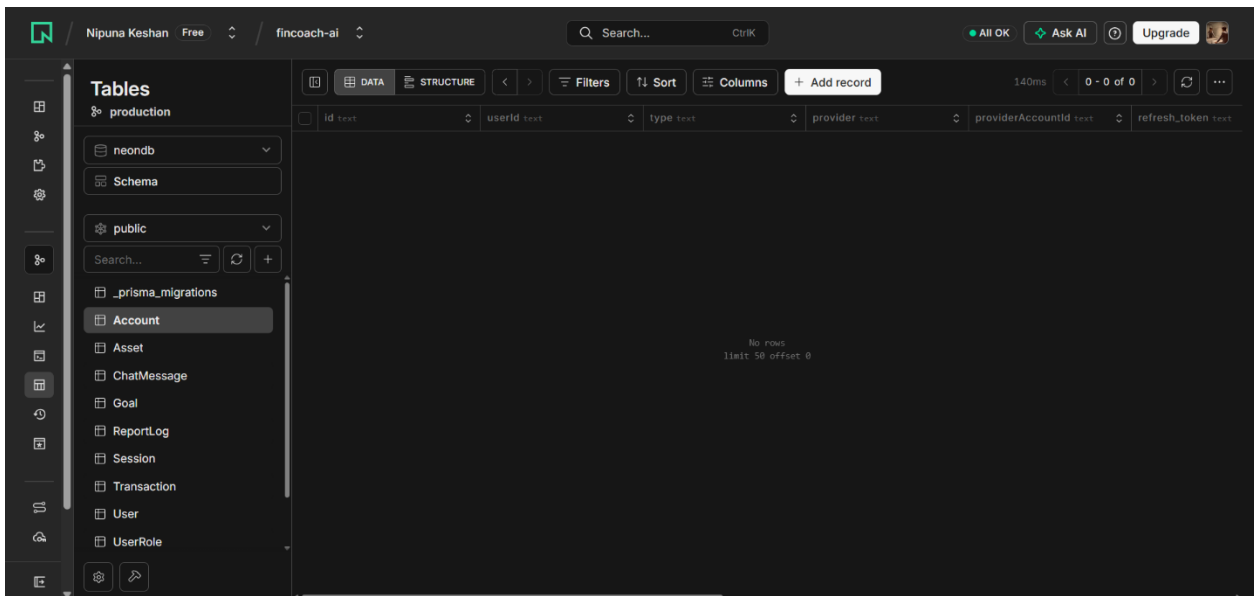


Figure 3.28: Account Table – OAuth/Credential Linkage

The Account table used by NextAuth.js to link a login method (e.g. Google) to a user. Its Primary Key is id, and userId is a Foreign Key referencing User.id. It was empty in this screenshot because the test account signed up with email/password rather than Google.

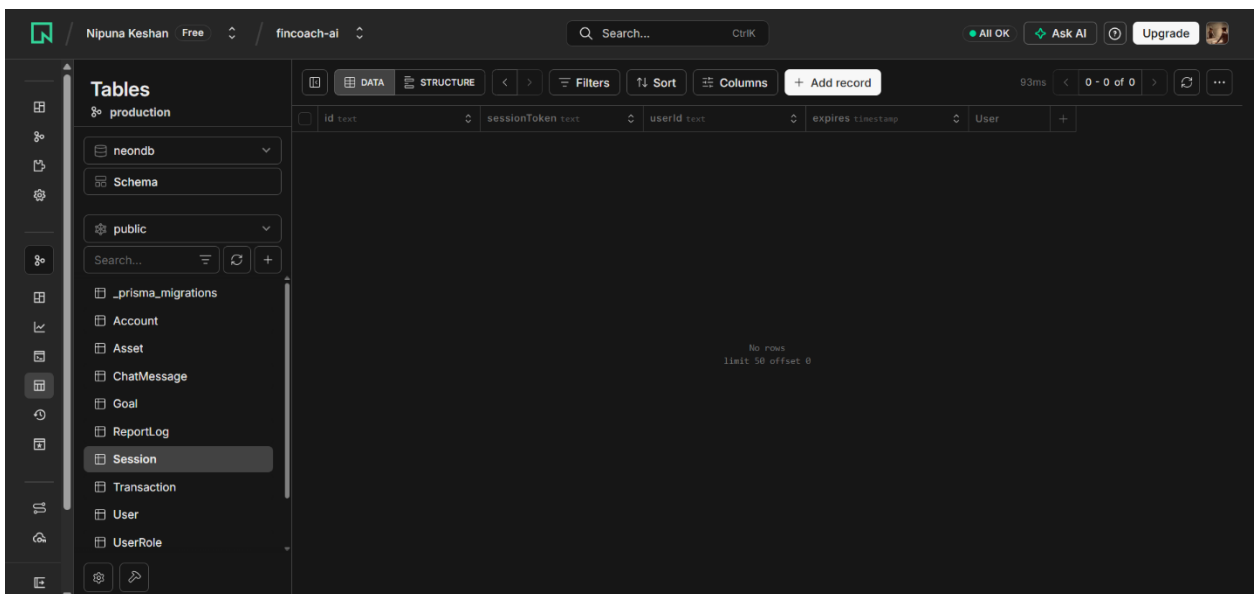


Figure 3.29: Session Table – Active Login Sessions

The Session table stores active login sessions. Its Primary Key is id, and userId is a Foreign Key referencing User.id. It was empty at the time of the screenshot because no session was active at that exact moment.

id	userid	type	amount	currency	amountBase
cmr12etnw000304jsd01x9...	cmr12aylm000004js2epmx...	expense	5000	LKR	5000
cmr12f43b000404jsv1hw8...	cmr12aylm000004js2epmx...	income	75000	LKR	75000
cmr12ffcz000504jsi62g3...	cmr12aylm000004js2epmx...	expense	15000	LKR	15000
cmr12fojl000604js0zwg6...	cmr12aylm000004js2epmx...	expense	2000	LKR	2000

Figure 3.30: Transaction Table – Income and Expense Records

Four real Transaction rows - a grocery expense, a salary income, a rent payment, and a takeaway expense. Primary Key: id. Foreign Key: userId referencing User.id, confirming every transaction belongs to exactly one user.

id	userid	name	targetAmount	currentAmount	currency
cmr12gbfe000704jshn4e0...	cmr12aylm000004js2epmx...	Buy Car	2000000	10000	LKR
cmr12gtl9000004jsdgiiv...	cmr12aylm000004js2epmx...	France Trip	500000	450000	LKR
cmr12hcs000004js0nm5k...	cmr12aylm000004js2epmx...	Emergency	100000	27000	LKR

Figure 3.31: Goal Table – Savings Goal Records

Three real savings goals - "Buy Car," "France Trip," and "Emergency" - each with a target and current amount. Primary Key: id. Foreign Key: userId referencing User.id.

id text	userId text	name text	category text	units text	value double precision
cmr12jo52000a04jsdj3l4...	cmr12aylm000004js2epmx...	FD	Bank / Fixed Deposit	1	100000
cmr12k3dj000b04js5jo11...	cmr12aylm000004js2epmx...	Bitcoin	Crypto	1	2500000
cmr12kqi000c04jsctvwg...	cmr12aylm000004js2epmx...	Gift	Other	1	25000

Figure 3.32: Asset Table – Portfolio Holdings

Three real investment records - a Fixed Deposit, Bitcoin, and a Gift - each with category and value. Primary Key: id. Foreign Key: userId referencing User.id.

id text	userId text	role text	content text	createdAt timestamp	User
cmr12ll3q000d04jshmxdd...	cmr12aylm000004js2epmx...	user	Mage me mase wiyadam k...	2026-07-12 17:31:30.71	User
cmr12lxzp000e04jsmctor...	cmr12aylm000004js2epmx...	assistant	Machan, oyage me mase ...	2026-07-12 17:31:47.413	User

Figure 3.33: ChatMessage Table – AI Coach Conversation History

Two real messages - one from the user and one AI reply - proving AI Coach conversations are saved. Primary Key: id. Foreign Key: userId referencing User.id. Note: this screenshot reflects an earlier single-table version of the chat schema; the model was later split into ChatSession and Message (Figure 3.25) so that a user's conversation history could be organised into separate, nameable threads.

Tables: production

neondb

Schema

public

Search...

._prisma_migrations

Account

Asset

ChatMessage

Goal

ReportLog

Session

Transaction

User

UserRole

DATA STRUCTURE Filters Sort Columns Add record

267ms 1 - 1 of 1

id text	userid text	month text	type text	sentAt timestamp	emailSentTo text
cmr12t4q000f04js7435L...	cmr12aylm000004js2epmx...	2026-04	combined	2026-07-12 17:37:22.728	nipunakshan2@gmail.com

Figure 3.34: ReportLog Table – Financial Report Delivery Log

One record of a financial report that was generated and emailed out, including the month it covered and the destination email. Primary Key: id. Foreign Key: userId referencing User.id.

Tables: production

neondb

Schema

public

Search...

Asset

ChatMessage

Goal

ReportLog

Session

Transaction

User

UserRole

UserRoleHistory

VerificationToken

DATA STRUCTURE Filters Sort Columns Add record

111ms 1 - 1 of 1

id text	userid text	roleName text	displayName text	emoji text	isPrimary boolean
cmr12eckc000104jsufpd7...	cmr12aylm000004js2epmx...	Doctor	Fixed Salary	EMPTY_STRING	TRUE

Figure 3.35: UserRole Table – Multi-Role Profile Data

One record showing "Doctor" set as the primary profession. Primary Key: id. Foreign Key: userId referencing User.id, confirming a user can hold multiple linked role records.

The screenshot shows a database management tool with a sidebar on the left listing various tables like Asset, ChatMessage, Goal, ReportLog, Session, Transaction, User, UserRole, UserTypeHistory, and VerificationToken. The main area displays the 'UserTypeHistory' table with the following data:

id	userId	fromType	toType	reason	changedAt
cmr12ecq7000204jskfqo7...	cmr12aylm000004js2epmx...	NULL	Doctor	Onboarding completed	2026-07-12 17:25

Figure 3.36: UserTypeHistory Table – Role Change Audit Log

One record logging that onboarding was completed and the role was set to "Doctor." Primary Key: id. Foreign Key: userId referencing User.id - this table is an audit trail of role changes over time.

3.3 Architectural Design

FinCoach AI follows a modern full-stack, serverless architecture built on the Next.js 15 App Router. The client (browser) communicates with Next.js Route Handlers under the `/api/` namespace over HTTPS. Server Components handle data-intensive rendering while Client Components manage interactive UI elements such as forms and charts.

On the backend, API routes validate each request's authenticated session and use Zod (Zod, 2024) for input validation before interacting with the PostgreSQL database through Prisma ORM. The AI Coach route builds an aggregated monthly financial-summary context (approximately 400 tokens) from the user's transaction and goal data, then forwards it to the Anthropic Claude API (Anthropic, 2024a) via the Vercel AI SDK, which streams the AI's response back to the browser using Server-Sent Events (SSE) for real-time display. Bill-scanning requests send an uploaded image to the Claude API for AI-based OCR extraction, and the parsed results are returned for user confirmation before being saved as a transaction.

Currency conversion requests are fetched from the ExchangeRate-API and cached to minimise latency and external API calls. Email-based interactions — OTP verification codes and financial reports — are dispatched using Nodemailer over Gmail SMTP, authenticated with a Gmail App Password instead of a third-party transactional email provider, with monthly automated reports triggered by a Vercel Cron Job scheduled for the last day of each month. The entire application is deployed on Vercel, which provides serverless auto-scaling, edge-network delivery, and integrated cron-job scheduling. The main system actors and their interactions with these modules are illustrated in the use case diagram in Figure 3.37.

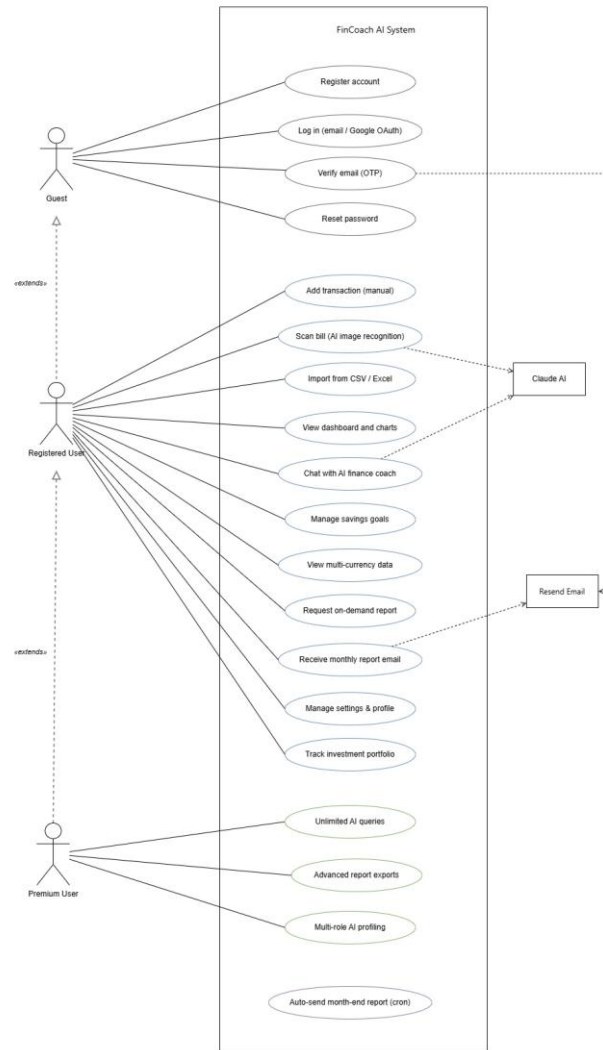


Figure 3.37: Use Case Diagram

Use case diagram showing Guest, Registered User, and Premium User actors and their interactions with the FinCoach AI system, including the external Claude AI service. The diagram also shows "Auto-send month-end report (cron)" as a standalone use case with no human actor, reflecting the Vercel Cron Job described in Section 3.3 that triggers automated monthly reports on a schedule rather than in response to a user action. Note: this diagram was produced during the design phase and labels the email service as "Resend" — the implemented system instead sends email through Gmail SMTP (via Nodemailer with a Gmail App Password).

Chapter 04 – Implementation

4.1 Development Approach

Building FinCoach AI fell to a single developer, so an Agile-Incremental approach made the most sense: short weekly cycles, each one aimed at a single working feature rather than a slice of many half-finished ones. Authentication and transactions came first, since almost everything else in the app depends on a signed-in user and a working data model. The AI-powered features followed once that foundation was solid enough to build on.

Every change went through Git and was pushed to GitHub, and the app was kept continuously deployed to Vercel throughout the eight weeks. This meant new features could be checked against the live production environment almost as soon as they were written, rather than only ever being tested locally.

4.2 Implementation of Key Modules

A brief account of how each major module from Chapter 02 came together follows.

Authentication was built on NextAuth.js v5, supporting both email/password login and Google OAuth. New accounts are not considered active until a 6-digit OTP, emailed through Nodemailer over Gmail SMTP, has been verified; the code itself is hashed and stored with an expiry in the VerificationToken table so it can be checked server-side rather than trusted at face value.

For transaction management, one API route handles all incoming data, running it through Zod validation before it ever reaches the Transaction table. Whether a transaction started as a manual entry, a scanned bill, or a row in an imported spreadsheet, it ends up in the same shape once it hits the database — which keeps the analytics and AI context logic simple, since neither has to care where a transaction originally came from.

The AI Coach works by pulling together a signed-in user's transactions and goals for the relevant month into a fairly small JSON summary, around 400 tokens, which is then sent to the Claude API through the Vercel AI SDK as context. Replies are streamed back over Server-Sent Events, so the answer appears gradually rather than all at once after a delay.

Bill scanning takes a slightly different route: the receipt photo goes straight to the Claude API, which reads it and returns a merchant name, amount, date, and category. Nothing is saved

automatically, though — the extracted details are shown to the user first, so a misread total or the wrong category can be fixed before it becomes a permanent record.

Multi-currency support relies on cached rates from the ExchangeRate-API, refreshed periodically rather than on every request to avoid hammering the external service. Each transaction stores its original amount and currency alongside a separately calculated amountBase figure in the user's preferred currency, so dashboard totals can simply be summed rather than converted on every page load.

Savings goals and portfolio tracking share more or less the same shape under the hood: create, update, delete against the Goal and Asset tables, with progress percentages and profit/loss worked out server-side, and a short AI-generated comment attached — a completion estimate for goals, a concentration warning for the portfolio.

Automated reporting is handled by a Vercel Cron Job that runs on the last day of each month, checking which users have autoReportEnabled turned on, building their monthly summary, and emailing it out as formatted HTML through Nodemailer. Every send gets logged in ReportLog, both to give users a history of what they have received and because the same aggregation code is reused for the on-demand "Send Combined Report" button.

4.3 Challenges Faced

A few problems came up repeatedly enough during development to be worth recording here.

- Keeping the AI context small. Sending an entire transaction history to Claude on every chat message would be slow and expensive, so the fix was to aggregate everything into a monthly summary first and let that carry the detail instead.
- Making currency conversion behave the same way no matter how a transaction was entered. Manual entry, bill scanning, and file import all build a transaction differently under the hood, so the safest fix was to normalise everything to a common shape, with both the original and base-currency amounts stored, before it ever reaches the database.
- Sending email without a dedicated transactional provider. Gmail SMTP through Nodemailer worked well enough for Phase 1 traffic, though it is flagged as a limitation for anything beyond that in Section 6.2.

- Simply covering full-stack design, implementation, and documentation alone in eight weeks. The main defense against running out of time was ordering the modules in Section 4.2 so that each one unblocked the next — authentication and the schema first, AI-dependent features only once that groundwork existed.

Chapter 05 – Testing

5.1 Testing Approach

Rather than leaving testing for the end, each module was checked against its expected behavior as soon as it was built, and always in the deployed Vercel environment so results reflected production rather than a local setup that might behave differently. Three levels of testing were used.

- Unit-level checks on individual pieces of logic — currency conversion, OTP generation and expiry, and the progress-percentage calculation used for goals.
- Integration testing across API routes, confirming that Zod validation, Prisma writes, and response shapes all behaved correctly, including failure cases such as an invalid token or a malformed transaction payload.
- User Acceptance Testing, walking through complete journeys end to end: registering and verifying an OTP, completing onboarding, adding transactions through all three entry methods, chatting with the AI Coach, creating and updating a goal, and receiving a report email.

5.2 Test Cases

Table 5.1 lists a representative sample of test cases covering the functional requirements set out in Section 2.2, rather than every single case that was run.

Table 5.1: Representative Test Cases

Test ID	Module	Test Description	Expected Result	Status
TC-01	Authentication	Register with a valid email and complete OTP verification	Account created; user redirected to onboarding	Pass
TC-02	Authentication	Enter an incorrect or expired OTP code	Verification rejected with a clear error message	Pass
TC-03	Transactions	Add a manual expense transaction with valid fields	Transaction saved and reflected in the transaction list and dashboard totals	Pass
TC-04	Transactions	Upload a receipt image via Bill Scan	AI extracts amount, date and category, shown for confirmation before saving	Pass

Test ID	Module	Test Description	Expected Result	Status
TC-05	Multi-Currency	Add a transaction in a non-base currency	Amount converted and stored correctly in the user's base currency	Pass
TC-06	AI Coach	Ask "Why did I overspend last month?"	AI answers using the user's real aggregated data rather than generic advice	Pass
TC-07	Savings Goals	Create a goal and update its progress	Progress bar and status badge update to reflect the new amount	Pass
TC-08	Reports	Request an on-demand report for a past month	Report email delivered and logged in ReportLog	Pass
TC-09	Security	Attempt to access another user's transactions via a direct API request	Request rejected; only the authenticated user's own data is returned	Pass

5.3 User Acceptance Testing Summary

Every journey listed above was completed successfully, with the AI Coach, bill-scanning OCR, and automated report emails all checked against real data in the live environment rather than mocked responses. A handful of minor issues turned up along the way - small rounding differences in multi-currency displays, and a timing edge case in OTP resends - and were fixed before the Phase 1 deadline. Nothing blocking remained outstanding by the time the project was submitted.

Chapter 06 – Results and Evaluation

6.1 Achievement of Objectives

Section 1.4 set out six objectives for Phase 1. All six were met, and each is functioning in the live application rather than existing only on paper.

- A working AI-powered financial coaching platform was built with Next.js 15 and the Claude API, and is deployed and running on Vercel.
- Transactions can be recorded through manual entry, AI-powered bill scanning, or CSV/Excel import, all of which are normalized into a single underlying model.
- The conversational AI chat gives personalized spending insights and goal-based advice, grounded in a user's actual monthly data rather than generic templated responses.
- Multi-currency support covers 32 currencies with live rates from the ExchangeRate-API, converted automatically to each user's base currency.
- Monthly reports go out automatically by email through a Vercel Cron Job, with an on-demand option available whenever a user wants one sooner.
- Savings goals are tracked with AI-generated progress advice and projected completion dates, shown as goal cards with live progress bars.

6.2 Limitations

Built solo over eight weeks, FinCoach AI naturally has some rough edges worth being upfront about.

- Email still runs through Gmail SMTP with an App Password rather than a dedicated transactional provider. That was fine for Phase 1 traffic, but a service such as Resend or SendGrid would likely be needed if the user base grew.
- The AI Coach only sees an aggregated summary for the current month; it has no memory of past conversations between sessions. Section 7.2 covers how Retrieval-Augmented Generation could address this.
- Portfolio values are entered by hand rather than pulled automatically from a bank, brokerage, or crypto exchange.

- There is no billing integration yet, so the "plan" field on the User model exists in the schema without being enforced against any actual payment provider.
- Testing, described in Chapter 05, was carried out manually by the developer rather than through an automated suite or a separate QA process — reasonable for an academic Phase 1 project, but something a production launch would need to build on.
- Feedback so far has come mainly from the developer and project supervisor; the system has not yet been tried by a wider group of external users.

Chapter 07 – Conclusion and Future Work

7.1 Conclusion

The premise behind FinCoach AI was fairly simple: a finance tracker on its own leaves people to work out what their numbers mean, but a conversational layer grounded in that same data can turn it into something closer to actual advice. Across eight weeks, that premise was built out into a working full-stack application — authentication, transaction entry from three different sources, multi-currency support, AI coaching and bill scanning, savings goals, a portfolio tracker, and automated monthly reporting — all designed, implemented, tested, and deployed by one person.

Every objective set out in Section 1.4 was met, and the result is a live system running on Vercel with a production database on Neon rather than a prototype that only works on a local machine. Along the way, a few practical lessons stood out in particular: keeping AI context small and cost-efficient matters as much as getting the prompt right, and normalizing data from several different entry points early saves a lot of complexity later. Both of those lessons shape the future direction outlined below.

7.2 Future Work

The longer-term vision from Section 1.4 points beyond Phase 1 in a few directions.

- Retrieval-Augmented Generation (Lewis et al., 2020), to give the AI Coach persistent memory across sessions instead of relying purely on a single month's aggregated summary each time.
- Stripe subscription billing, which would give the existing "plan" field something real to enforce, and open up a paid tier with higher AI usage limits.
- Plaid bank-account synchronization, to replace manual entry and CSV import with transactions that arrive automatically from a user's actual bank.
- A native mobile application, since bill scanning and notifications in particular would benefit from being built for a phone rather than adapted from a responsive web layout.
- An automated test suite, using something like Jest or Playwright, to take over from the manual testing process in Chapter 05 as the codebase grows.

Taken together, these would move FinCoach AI on from a Phase 1 MVP towards the fuller personal finance ecosystem it was originally envisioned as.

Appendices

Appendix A: List of Abbreviations

Table A.1: Abbreviations Used in This Report

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
ERD	Entity Relationship Diagram
FK	Foreign Key
HND	Higher National Diploma
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
MVP	Minimum Viable Product
OAuth	Open Authorization
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
OTP	One-Time Password
PK	Primary Key
RAG	Retrieval-Augmented Generation
SaaS	Software as a Service
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
SSE	Server-Sent Events
UAT	User Acceptance Testing

Appendix B: Software Requirements Specification Reference

A fuller Software Requirements Specification was produced separately, following the IEEE 830-1998 standard mentioned in Section 2.1, and covers the functional requirements, non-functional attributes, interface requirements, and design constraints in more depth than this report needs. Section 2.2 gives a condensed version of the functional requirements for readability; the full SRS is kept alongside the project's source repository.

References

- 1) Anthropic. (2024a). *Claude API reference*. <https://docs.anthropic.com>
- 2) Anthropic. (2024b). *Claude model overview*. <https://docs.anthropic.com>
- 3) Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). *Language models are few-shot learners*. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- 4) Chen, Y., Jiang, B., & Li, H. (2021). *Understanding user abandonment in personal finance management applications: A mixed-methods study*. *Journal of Financial Technology*.
- 5) ExchangeRate-API. (2024). *Exchange rates API documentation*. <https://www.exchangerate-api.com/docs>
- 6) Fernandes, D., Lynch, J. G., Jr., & Netemeyer, R. G. (2014). Financial literacy, financial education, and downstream financial behaviors. *Management Science*, 60(8), 1861–1883. <https://doi.org/10.1287/mnsc.2013.1849>
- 7) Google. (2024). *Sign in with app passwords*. Google Account Help. <https://support.google.com/accounts/answer/185833>
- 8) Institute of Electrical and Electronics Engineers. (1998). *IEEE Std 830-1998: IEEE recommended practice for software requirements specifications*. IEEE.
- 9) Intuit Inc. (2006). *Mint: Personal finance and money management*. <https://mint.intuit.com>
- 10) Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. *Advances in Neural Information Processing Systems*, 33. <https://arxiv.org/abs/2005.11401>
- 11) Neon. (2024). *Neon serverless PostgreSQL: Technical documentation*. <https://neon.tech/docs>

- 12) Next.js. (2024). *App Router documentation (Version 15)*. Vercel Inc. <https://nextjs.org/docs/app>
- 13) NextAuth.js. (2024). *NextAuth.js documentation*. <https://authjs.dev>
- 14) Nodemailer. (2024). *Nodemailer documentation*. <https://nodemailer.com/about/>
- 15) Prisma. (2024). *Prisma documentation*. <https://www.prisma.io/docs>
- 16) Quicken Inc. (1983). *Quicken personal finance software*. <https://www.quicken.com>
- 17) shadcn/ui. (2024). *shadcn/ui component library*. <https://ui.shadcn.com>
- 18) Tailwind Labs. (2024). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>
- 19) Vercel Inc. (2024a). *Vercel AI SDK documentation*. <https://sdk.vercel.ai/docs>
- 20) Vercel Inc. (2024b). *Vercel deployment documentation*. <https://vercel.com/docs>
- 21) Yang, Z., Liu, J., & Wang, K. (2023). LLMs for financial advisory: Evaluating contextual accuracy and hallucination rates in structured-context prompting. *Proceedings of the 2023 International Conference on AI in Finance*, 112–128.
- 22) YNAB. (2004). *You Need A Budget: Personal budgeting software*. <https://www.youneedabudget.com>
- 23) Zod. (2024). *Zod validation library*. <https://zod.dev>